

Algorithms and data structures I.

Heap
Priority Queue
HeapSort

Tree structures

- ◆ **Proper (or full) binary tree:**

Every non-leaf (internal) node have exactly 2 children.

- ◆ **Perfect binary tree:**

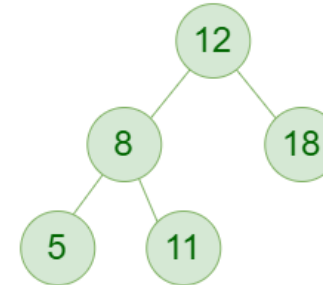
A proper (or full) binary tree, which has every leaf on the same level (depth)

- ◆ **Complete binary tree:**

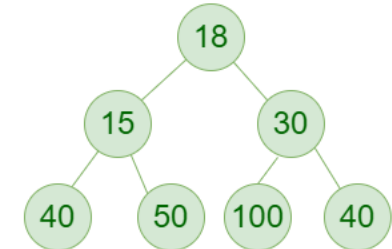
A perfect binary tree, from which we take away some leaves from the lowest level, specifically such that leaves might be missing „from the right”. Rigorously one could say that we cannot insert a new node such that it becomes the left child of another leaf.

If all levels of the binary tree are filled, except possibly the last one and the nodes on the last level are filled from left to right.

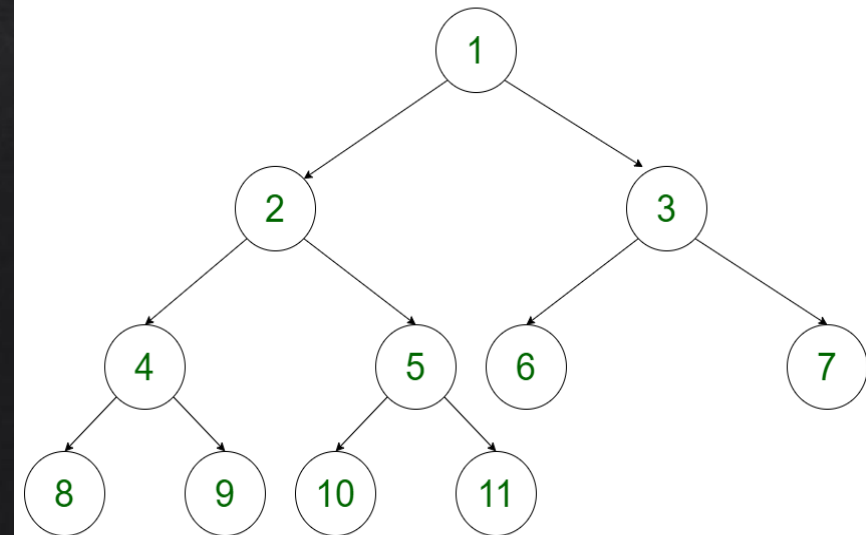
Full Binary Tree



Perfect Binary Tree



Complete Binary Tree



Tree structures

- ◆ **Proper (or full) binary tree:**

Every non-leaf (internal) node have exactly 2 children.

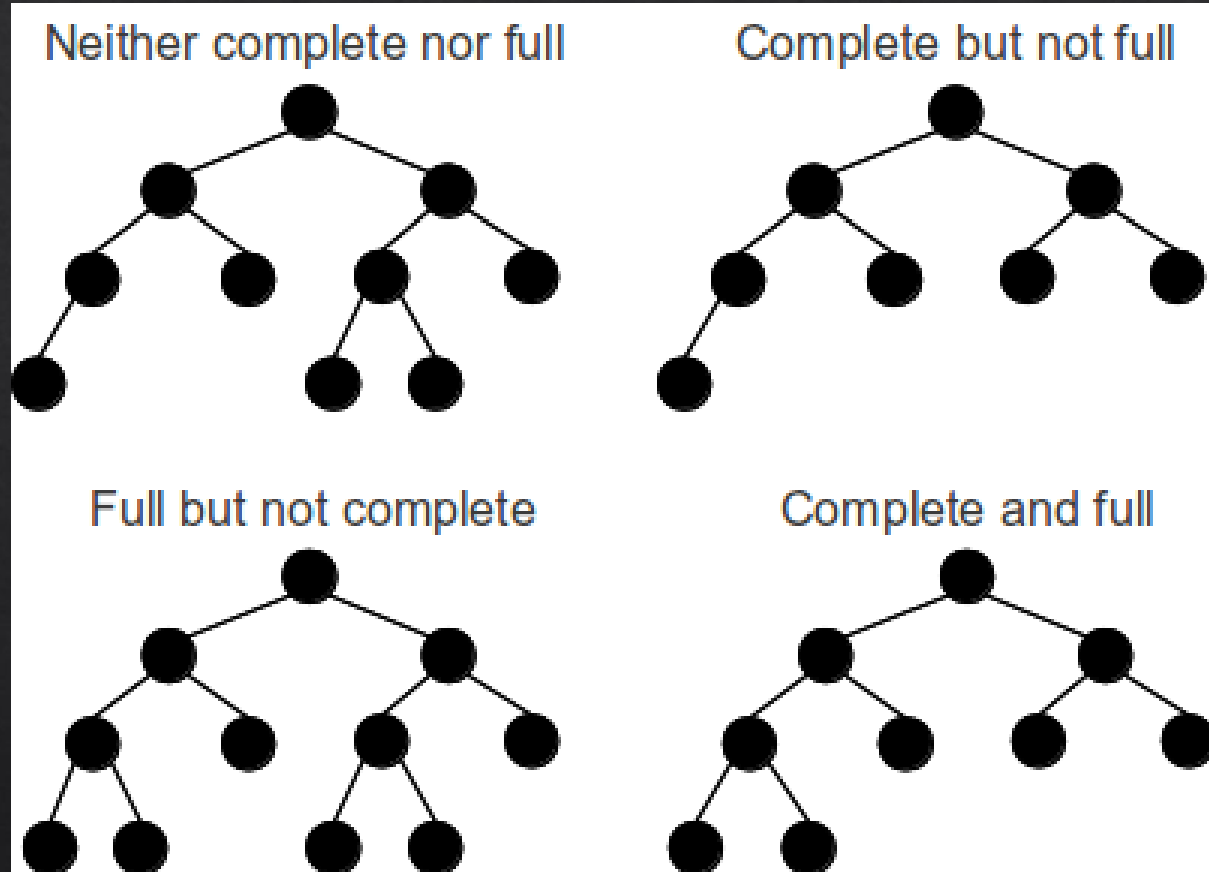
- ◆ **Perfect binary tree:**

A proper (or full) binary tree, which has every leaf on the same level (depth)

- ◆ **Complete binary tree:**

A perfect binary tree, from which we take away some leaves from the lowest level, specifically such that leaves might be missing „from the right”. Rigorously one could say that we cannot insert a new node such that it becomes the left child of another leaf.

If all levels of the binary tree are filled, except possibly the last one and the nodes on the last level are filled from left to right.

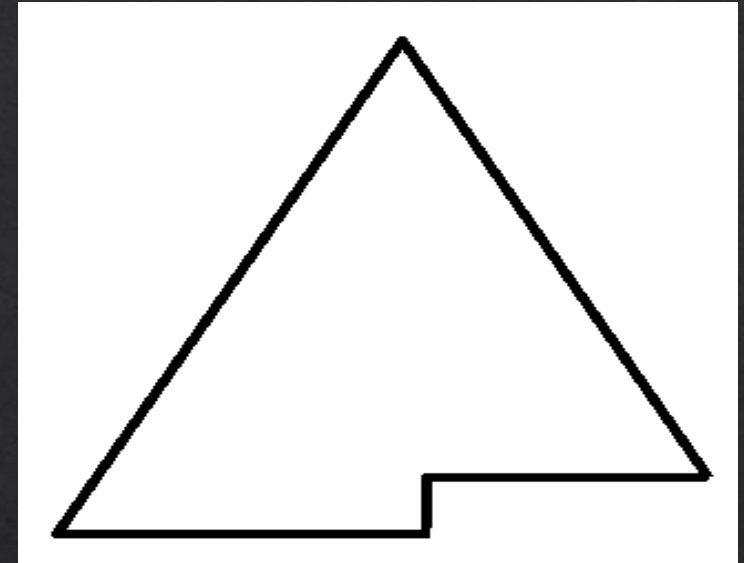


Heap

- ◆ **Complete bintree:**

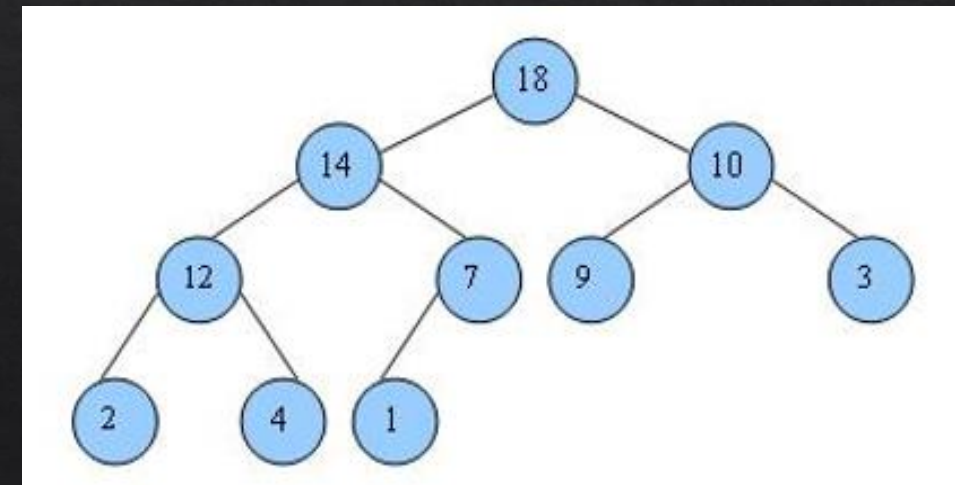
A perfect binary tree, from which we take away some leaves from the lowest level, specifically such that leaves might be missing „from the right”. Rigorously one could say that we cannot insert a new node such that it becomes the left child of another leaf.

If all levels of the binary tree are filled, except possibly the last one and the nodes on the last level are filled from left to right.



heap: *maximum or minimum heaps.* **Maximum heap:** A binary maximum heap is a complete binary tree where the key stored in each node is greater than or equal to ($\geq$) the keys in the node's children.

- ◆ There can be equal key values in a heap.
- ◆ This means, that we can reach the maximum element directly (it is in the root)
- ◆ Minimum heaps can be defined analogously
- ◆ The height of a complete binary tree with n nodes is $\lfloor \log n \rfloor$



Arithmetic representation

- ◆ Heaps (and complete binary trees) can be efficiently represented with an array, since level-order traversal gives us a structural enumeration of elements which is unambiguous and has no gaps (this is called the arithmetic representation of this binary tree).

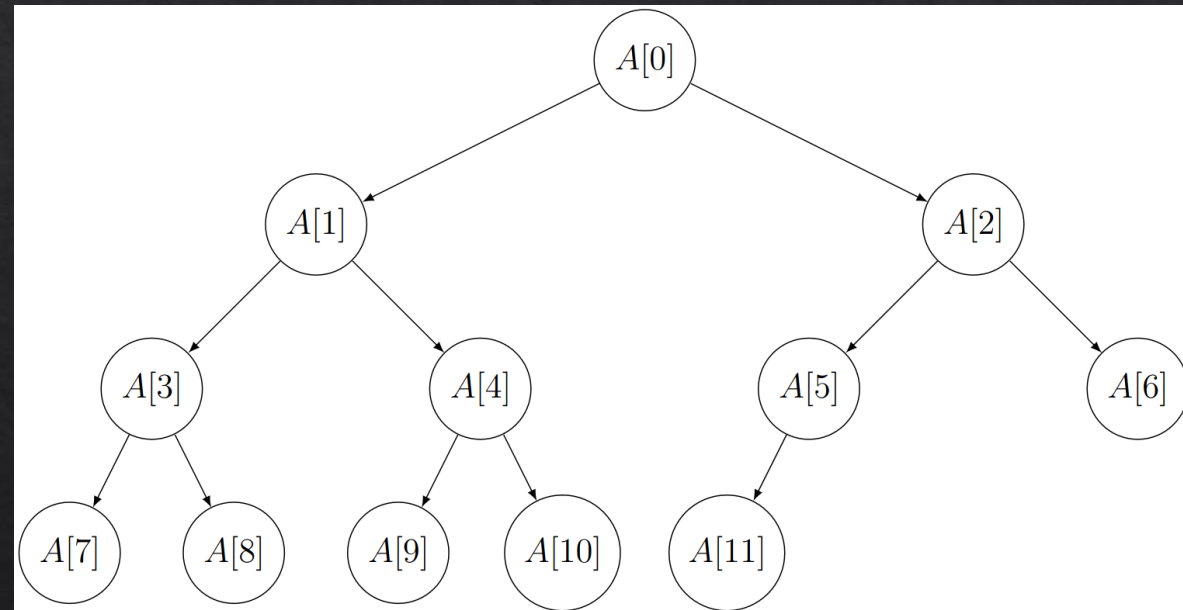
- ◆ We start indexing the array from 0.

- ◆ Index of the given node: i

- ◆ Index of left child: $2*i+1$

- ◆ Index of right child: $2*i+2$

- ◆ Index of parent: $\left\lfloor \frac{i-1}{2} \right\rfloor$

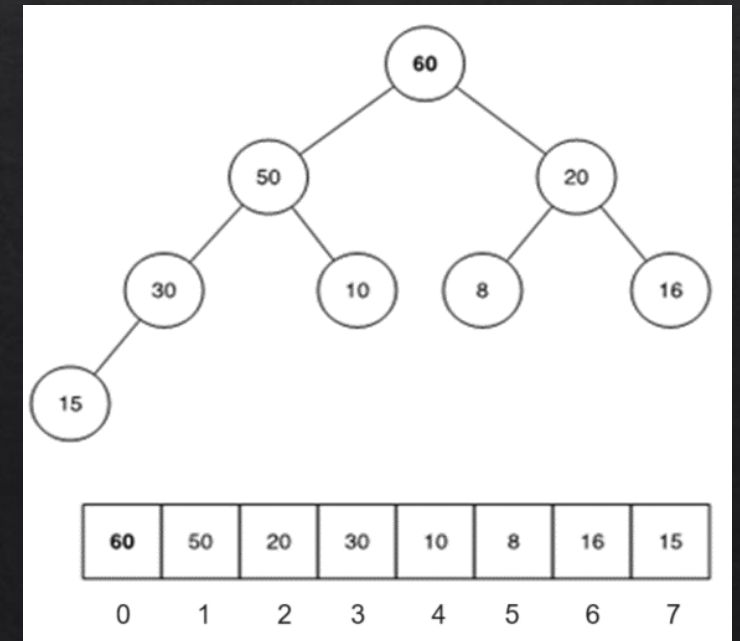
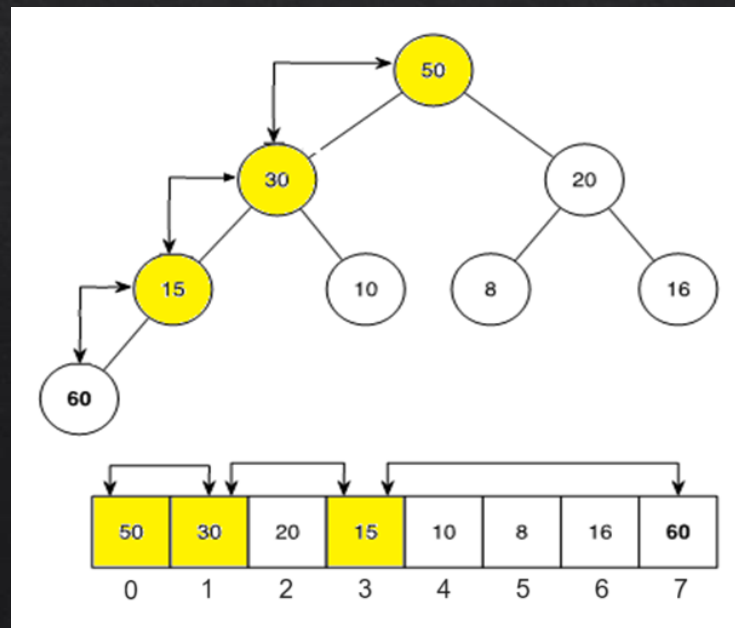
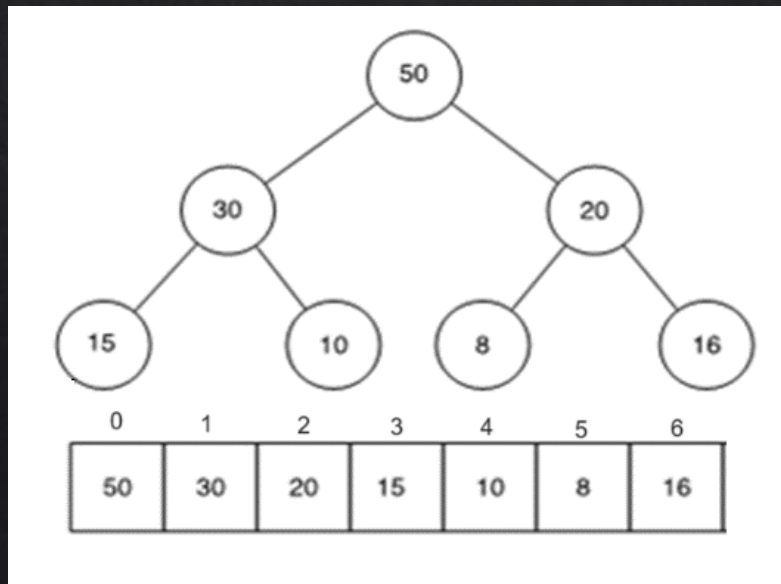


Heap operations

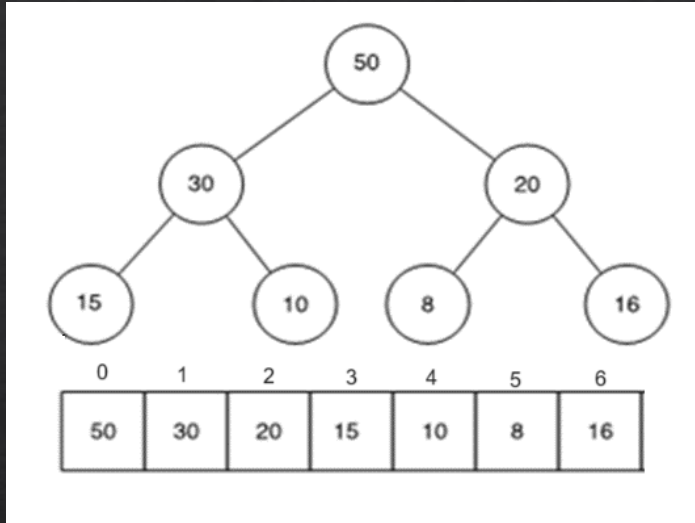
- ◆ Two main operations:
 - ◆ Insertion (add)
 - ◆ Removing maximum value (remMax)
- ◆ The heap is given in arithmetic representation: $A:T[n]$
- ◆ We will be viewing both the array A and the binary tree form of the heap.
- ◆ A heap usually does not occupy the whole array, there can be empty slots at the back, but there cannot be 'gaps'.
- ◆ Heaps can contain nodes with equal key values.

Insert

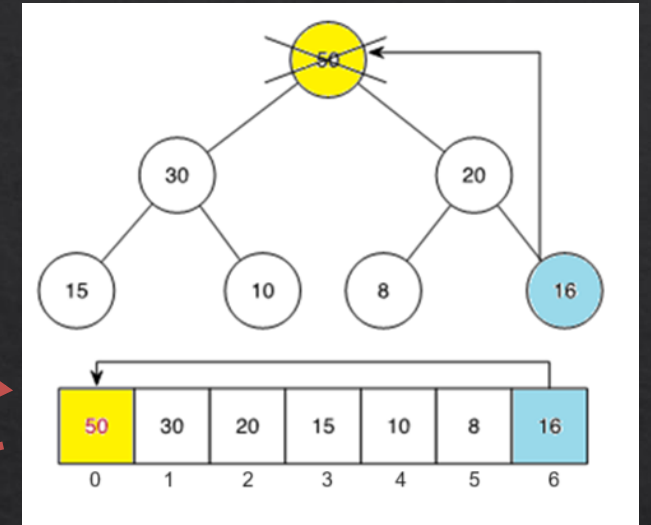
- ◆ Let's start with the heap (50, 30, 20, 15, 10, 8, 16, , ,). Insert 60 into this heap
- ◆ First step: we put 60 at the very end of the array.
- ◆ Second step: the new element **rises**, until the heap property becomes true, meaning we swap it with its parent until it is needed, or until we reach the top of the heap.
- ◆ This means at most $\log_2 n$ swaps since the height of the tree is $\text{floor}(\log_2 n)$



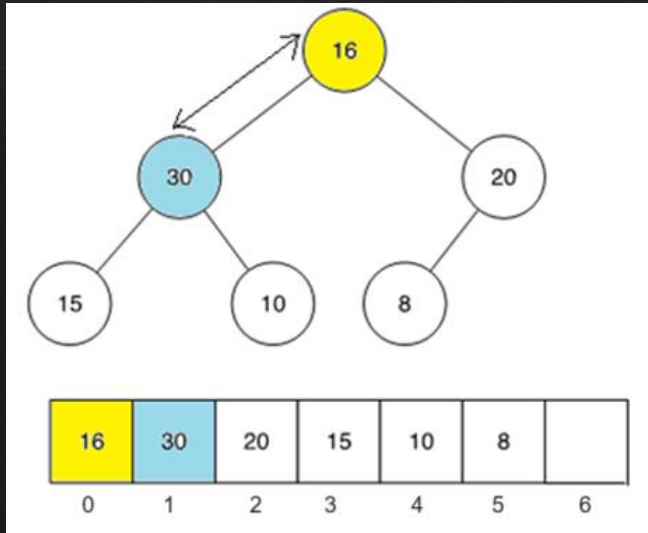
Deleting a node with max key value



- After removing the maximum element we put the last element from the array into its place. This preserves the continuity of the array.

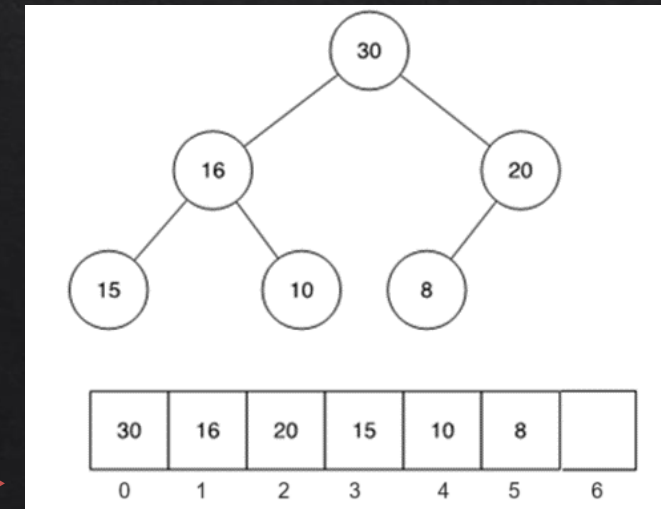


- Then the element begins **sinking**: As long as the element is smaller than any of its children we swap it with its larger child



- So we compare it with its children,
 - if larger, than we stop,
 - if smaller than one, then we swap it with the larger, and go on until there are no more children.

- Maximum number of swaps: $\text{floor}(\log_2 n)$



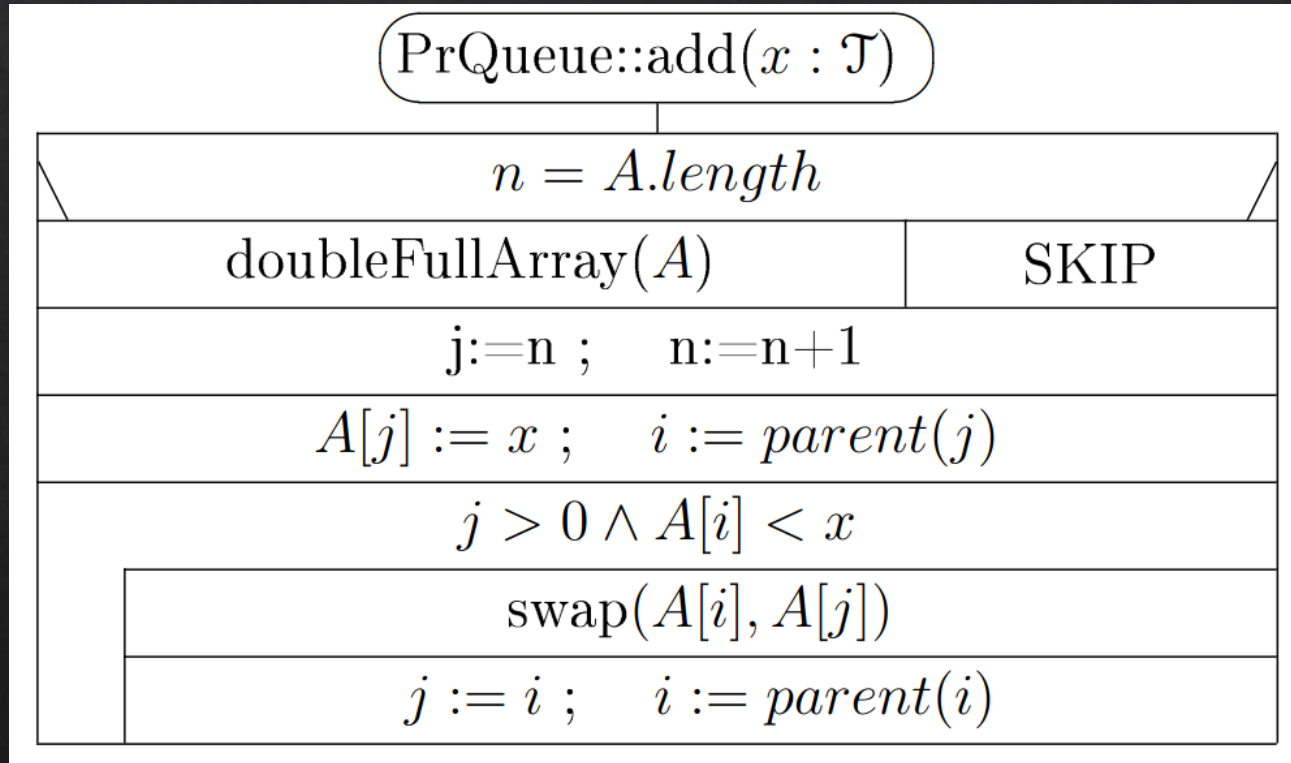
Priority Queue

- ◇ A priority queue is a queue where we can remove the largest element only (or smallest if it is a min priority queue)
- ◇ We can implement this with a heap.

PrQueue

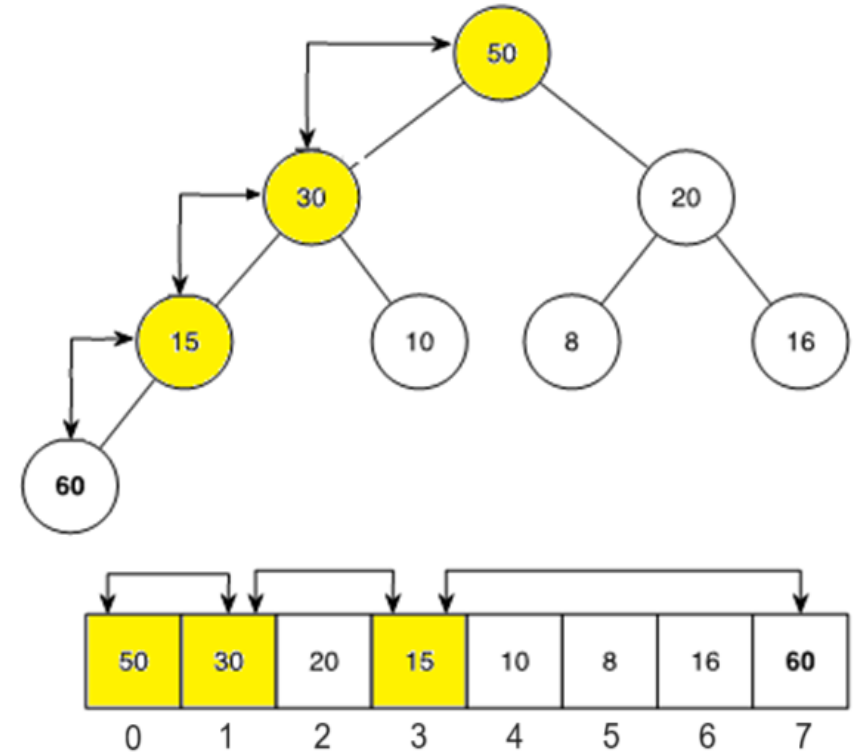
– $A : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the physical – constant $m0 : \mathbb{N}_+ := 16$ // size of the PrQ, its default is m0. – $n : \mathbb{N}$ // $n \in [0..A.length]$ is the actual length of the PrQ + $\text{PrQueue}(m : \mathbb{N}_+ := m0) \{ A := \mathbf{new} \mathcal{T}[m]; n := 0 \}$ // create an empty PrQ + $\text{add}(x : \mathcal{T})$ // insert x into the priority queue + $\text{remMax}():\mathcal{T}$ // remove and return the maximal element of the priority queue + $\text{max}():\mathcal{T}$ // return the maximal element of the priority queue + $\text{isEmpty}() : \mathbb{B} \{ \mathbf{return} \ n = 0 \}$ + $\sim \text{PrQueue}() \{ \mathbf{delete} \ A \}$ + $\text{setEmpty}() \{ n := 0 \}$ // reinitialize the priority queue

add(x:T)

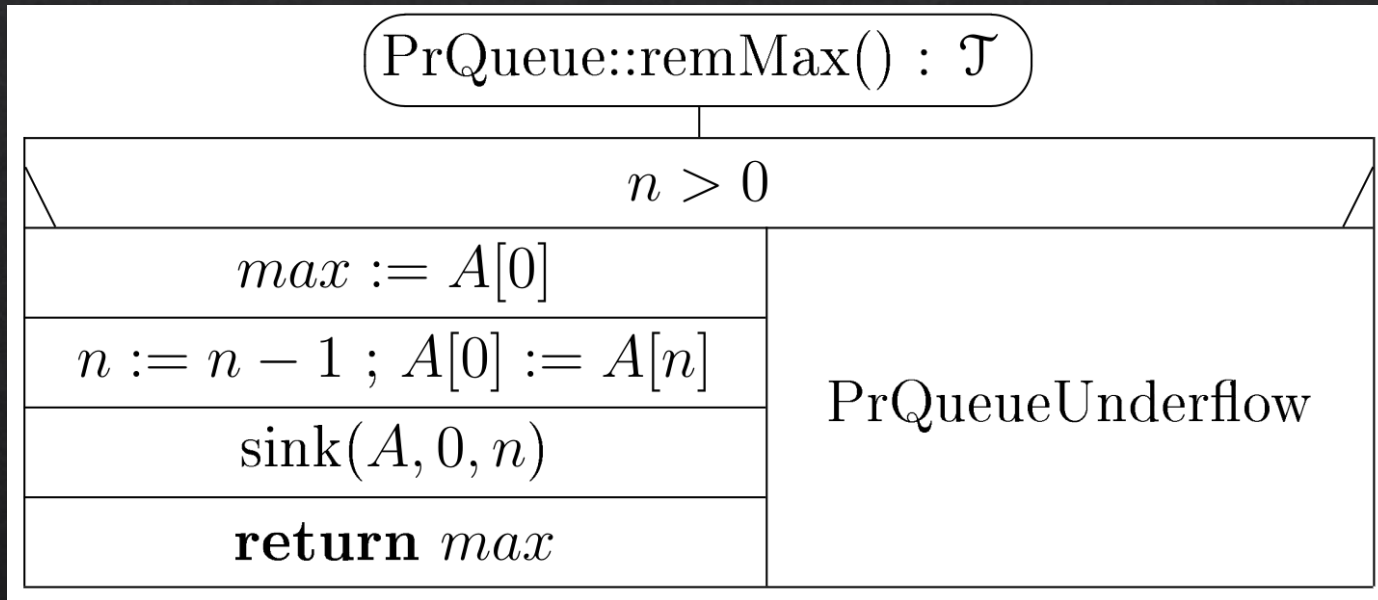


$$i := parent(j) = \left\lfloor \frac{j-1}{2} \right\rfloor$$

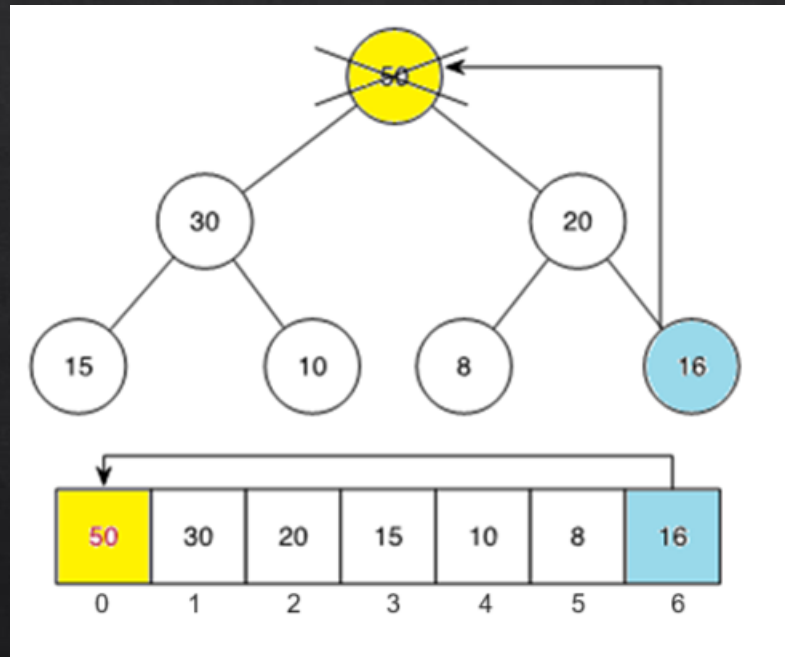
n=7, A.length=8, j=7 → n=8



remMax(): $\mathcal{T}$



$n=7 \rightarrow n=6$



max:=60
sink(A, 0, 6)
return 60

$\text{sink}(A:\mathcal{T}[];k,n:\mathbb{N})$ or down-heap operation

$\text{sink}(A:\mathcal{T}[];k,n:\mathbb{N})$

$i := k ; j := \text{left}(k) ; b := \text{true}$

$j < n \wedge b$

// $A[j]$ is the left child of $A[i]$

$j + 1 < n \wedge A[j + 1] > A[j]$

$j := j + 1$

SKIP

// $A[j]$ is the greater child of $A[i]$

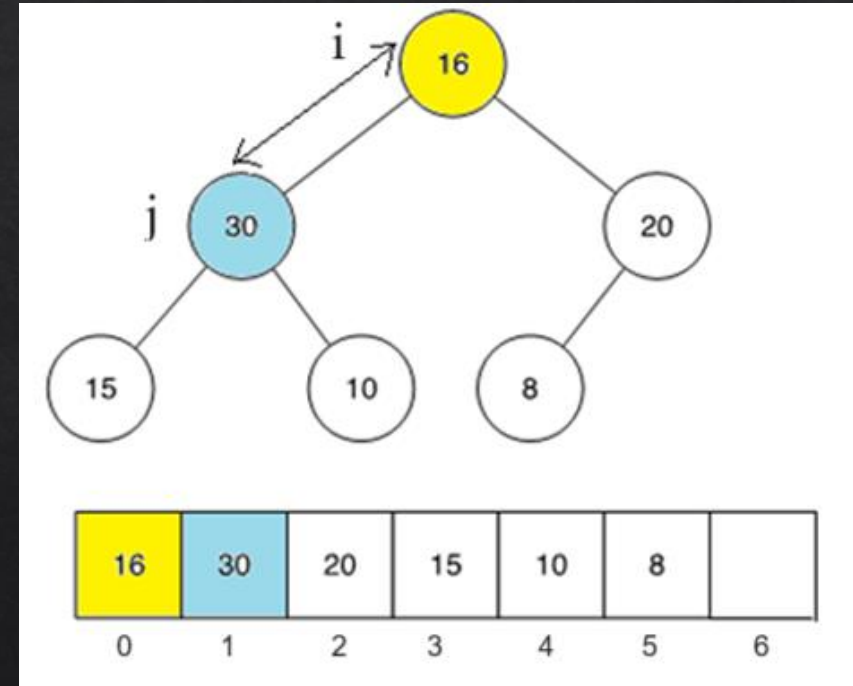
$A[i] < A[j]$

$\text{swap}(A[i], A[j])$

$i := j ; j := \text{left}(j)$

$b := \text{false}$

$\text{sink}(A,0,6)$

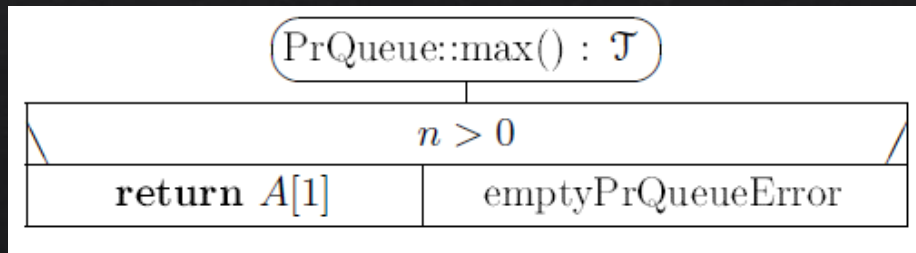


Implementations of priority queue

Possible implementations:

- unordered array + a variable to show which is the max element
- ordered array (non-decreasingly)
- heap

Maximum prioritásos sor megvalósításainak összehasonlítása			
	add(x)	remMax()	max()
unordered array + a variable to show where the max is	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
ordered array (non-decreasing order)	$O(n)$	$\Theta(1)$	$\Theta(1)$
heap	$O(\log n)$	$O(\lg n)$	$\Theta(1)$

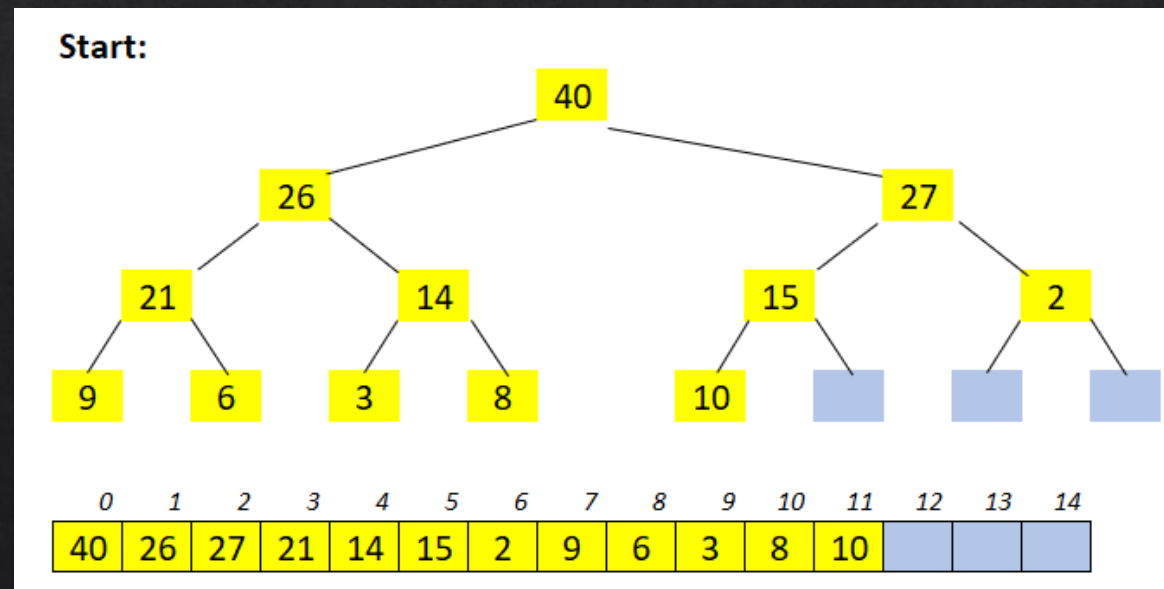


max(): $\mathcal{T}$

Exercise - heap:

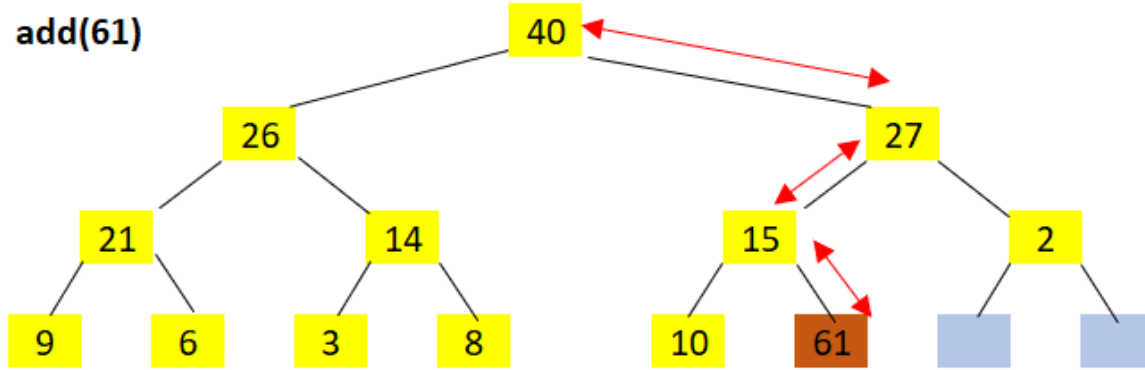
Exercise - heap:

- ◇ Q is a priority queue given by the the heap:
[40, 26, 27, 21, 14, 15, 2, 9, 6, 3, 8, 10, , ,].
- ◇ Present two **add(x)** procedures with key values: 61, 43.
- ◇ Present two **remMax()** procedures on the original heap (before insertion).
- ◇ Demonstrate these procedures in both the array and binary tree form.



◆ Q.add(61)

Insertion of 61:
add(61)



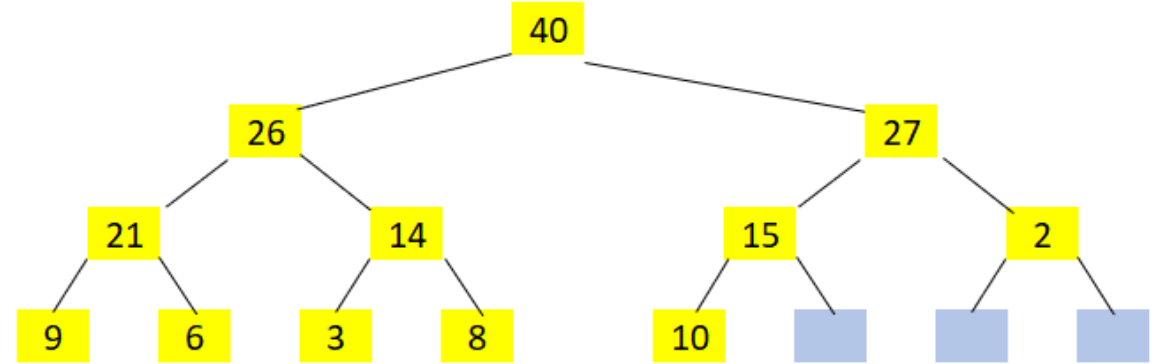
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
40	26	27	21	14	15	2	9	6	3	8	10	61		

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
40	26	27	21	14	61	2	9	6	3	8	10	15		

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
40	26	61	21	14	27	2	9	6	3	8	10	15		

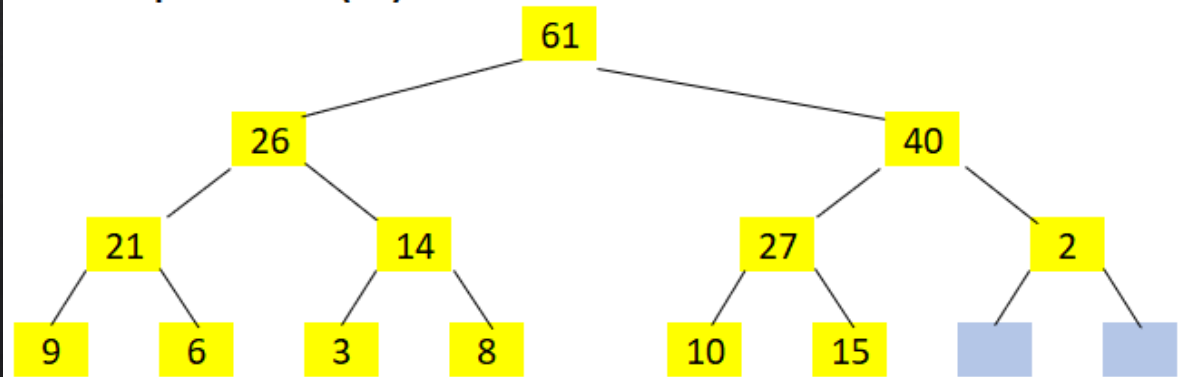
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
61	26	40	21	14	27	2	9	6	3	8	10	15		

Start:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
40	26	27	21	14	15	2	9	6	3	8	10			

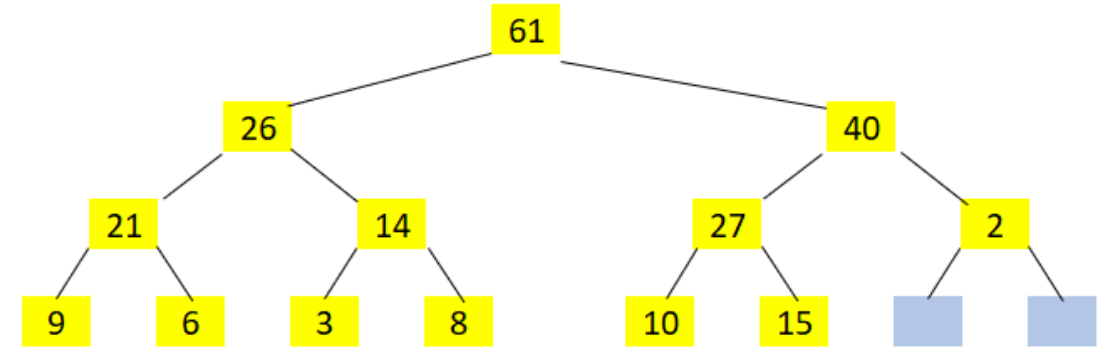
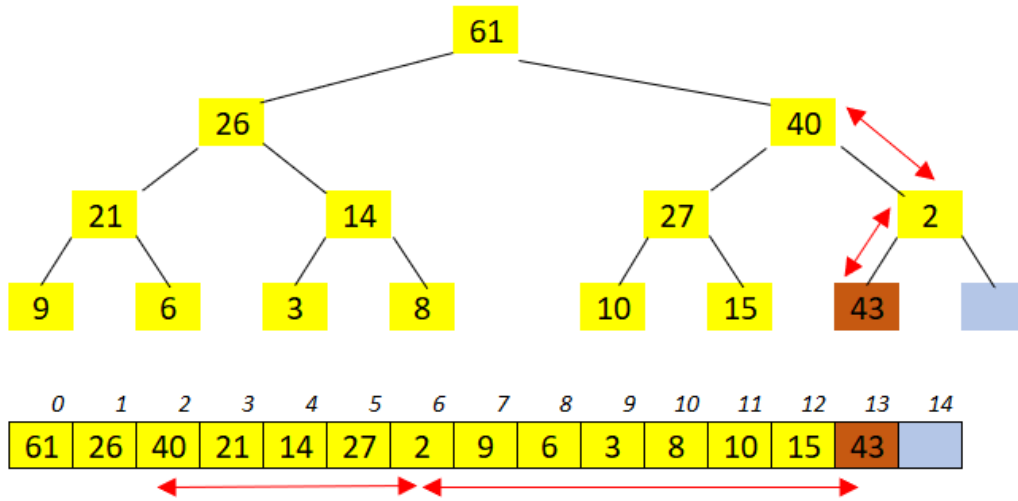
The heap after add(61):



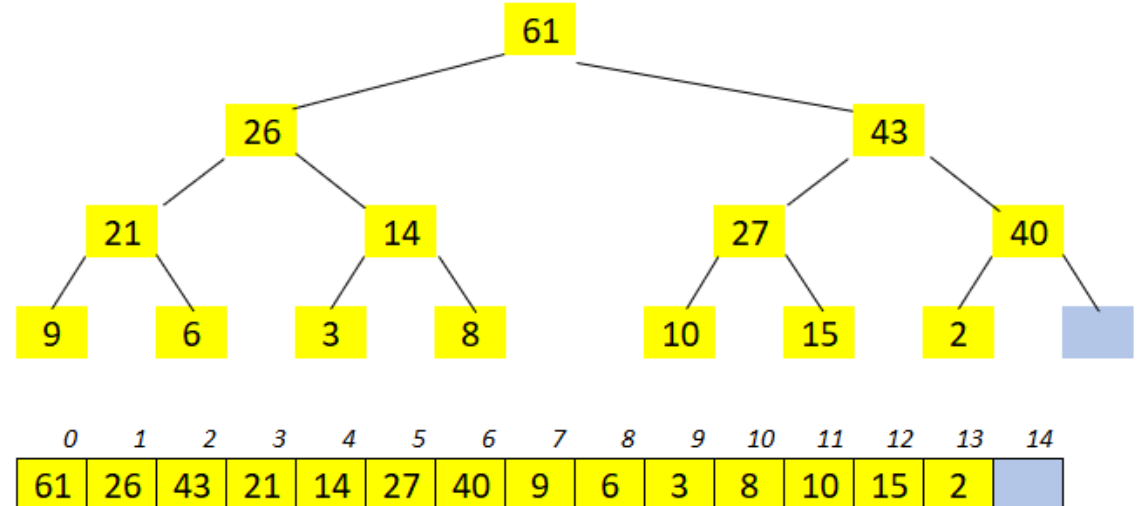
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
61	26	40	21	14	27	2	9	6	3	8	10	15		

◇ Q.add(43)

add(43):

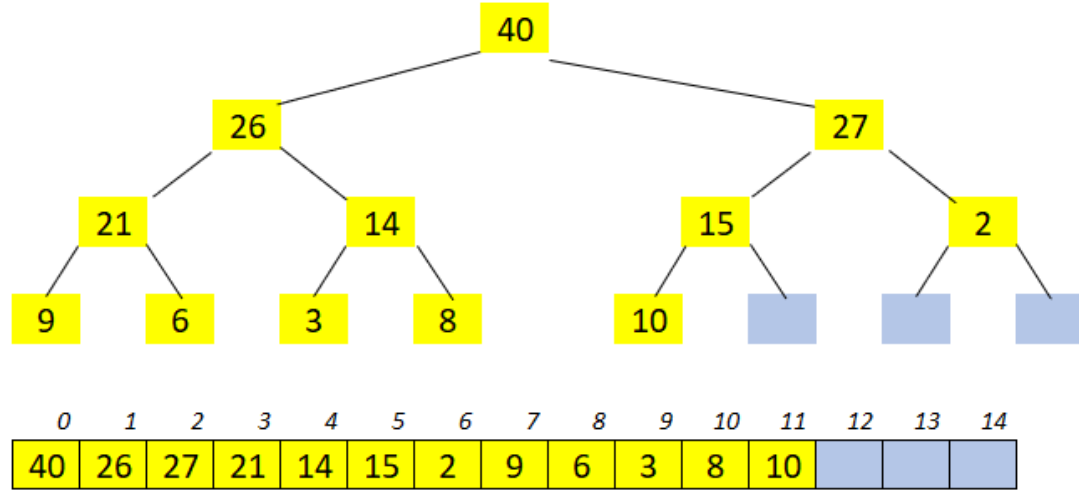


final heap:

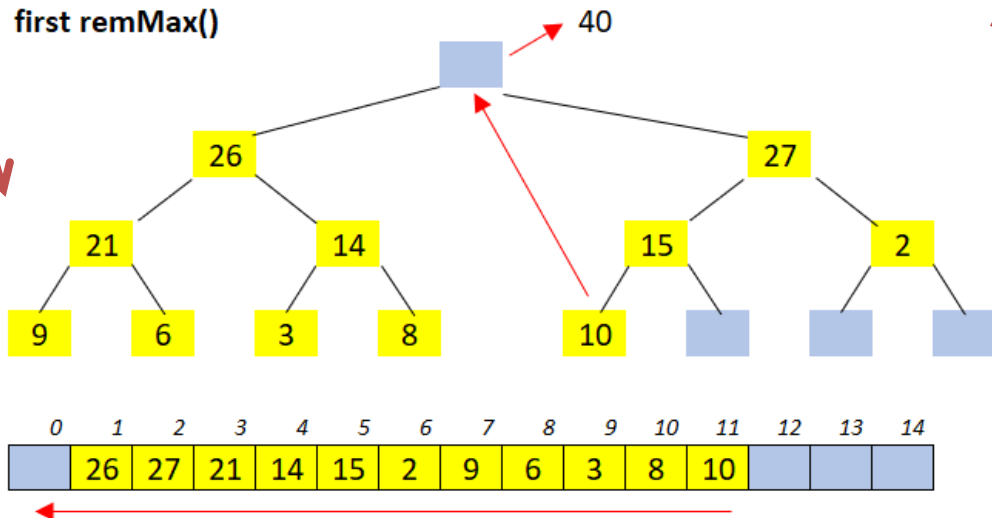


First Q.remMax():

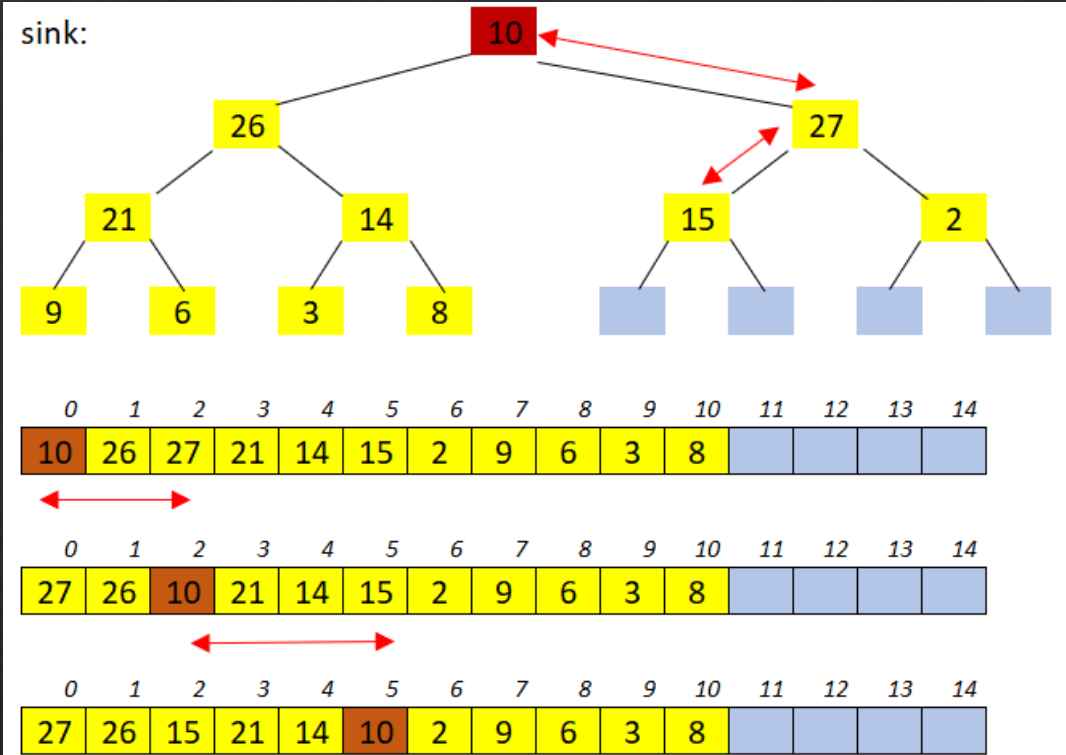
Start



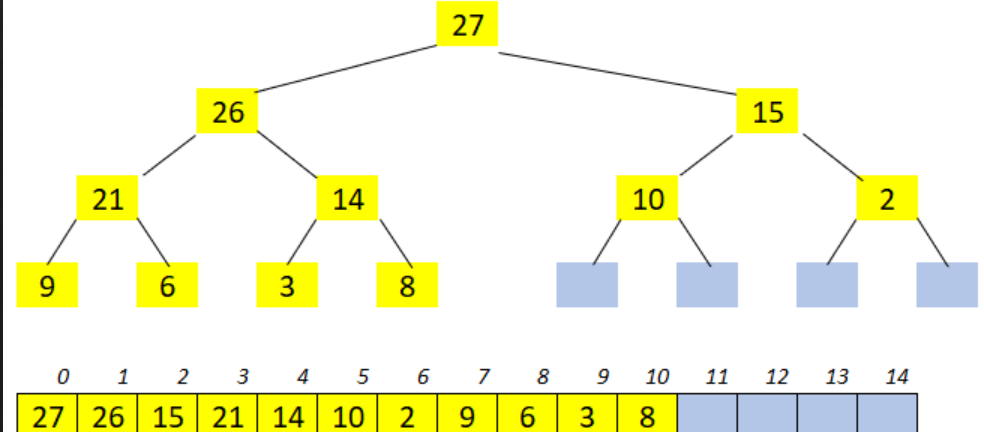
first remMax()



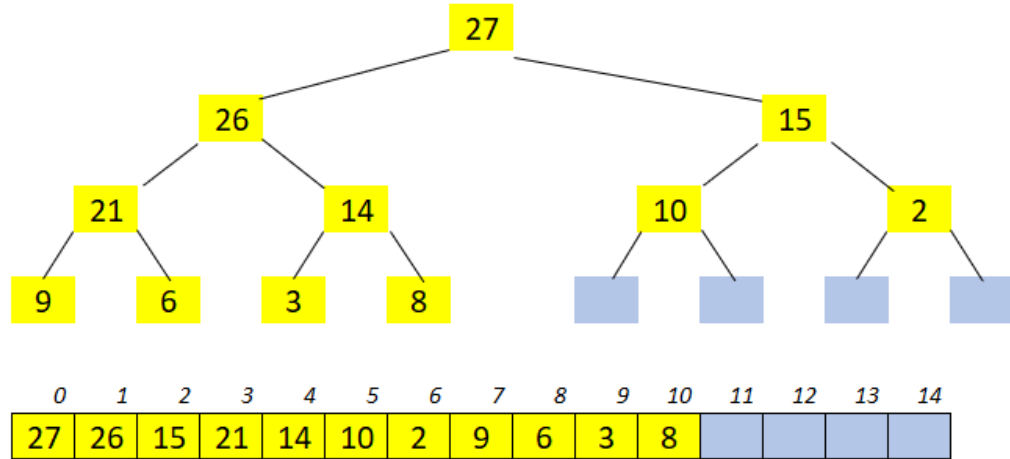
sink:



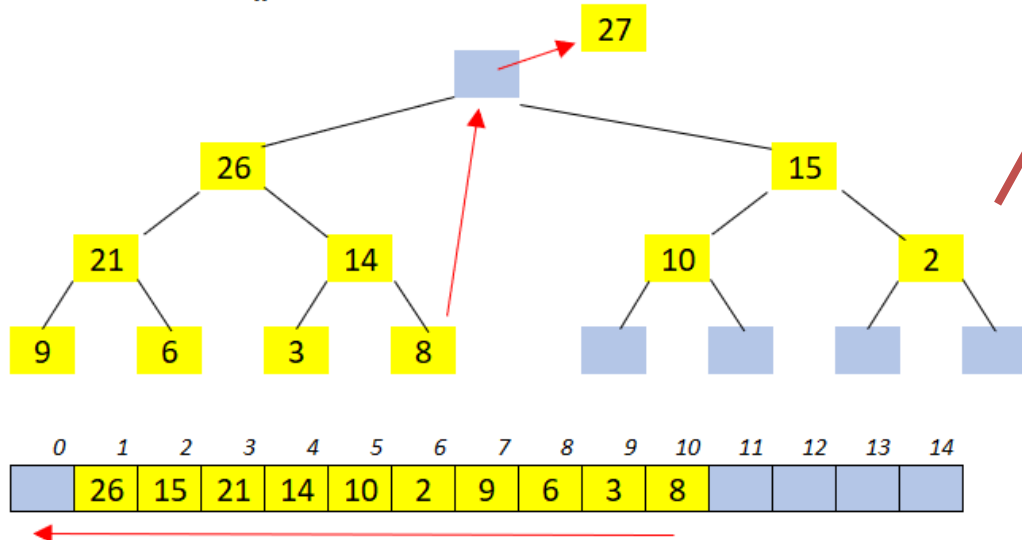
heap after first remMax():



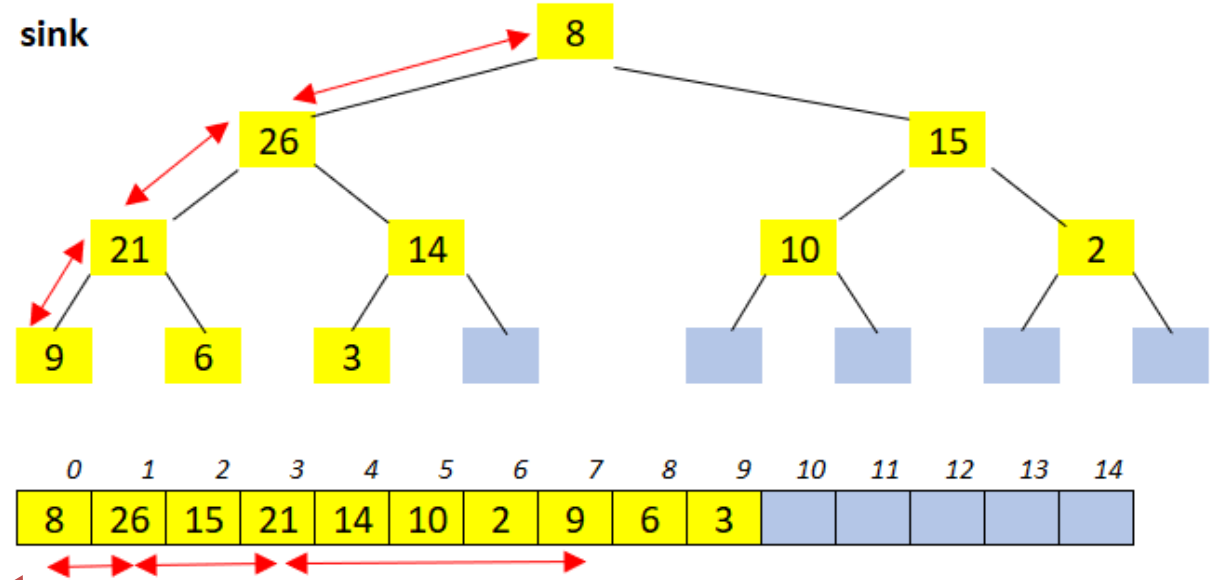
Second Q.remMax()



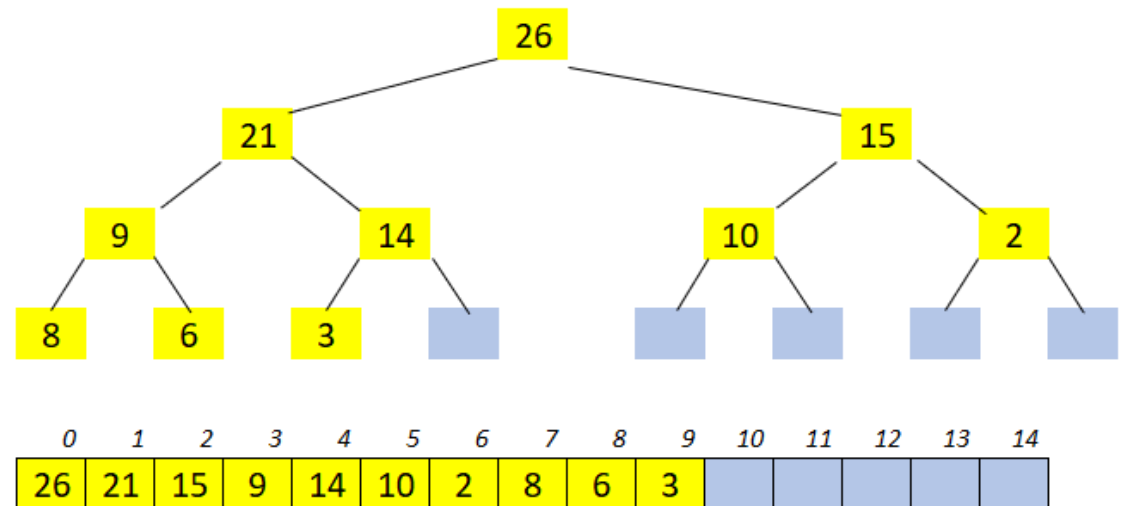
second remMax():



sink



heap after second remMax():



Sort with PrQueue

Sort with PrQueue

sortWithPrQueue($A : \mathcal{T}[n]$)

$Q : PrQueue(n)$

$i := 0$ to $n - 1$

$Q.add(A[i])$

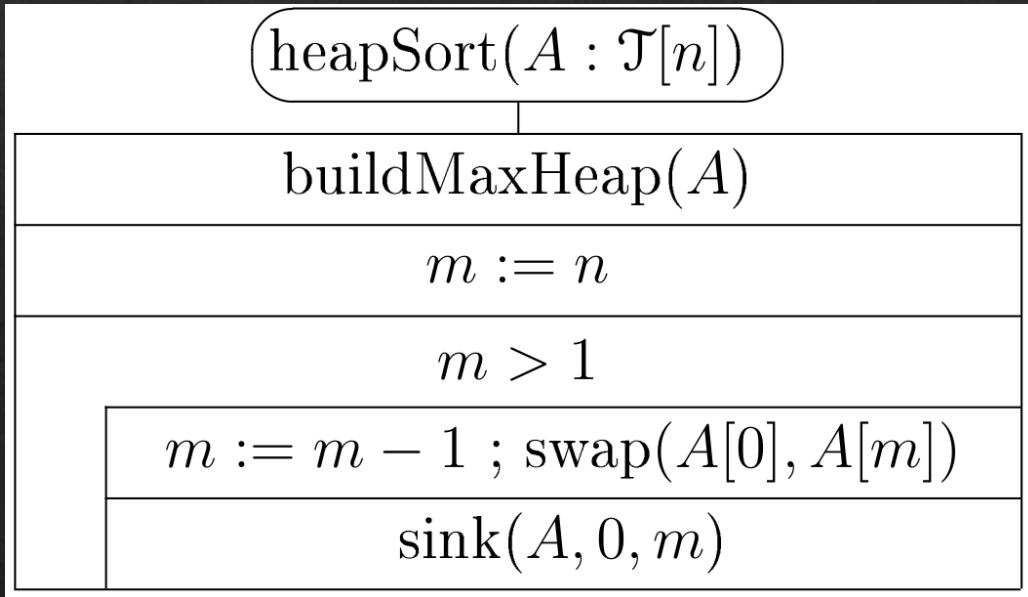
$i := n - 1$ downto 0

$A[i] := Q.remMax()$

Issue: Extra space
complexity

Heap sort

Heap sort



➤ First we take the array to be sorted and build a heap from it

➤ Next we take the elements and place them in their desired place

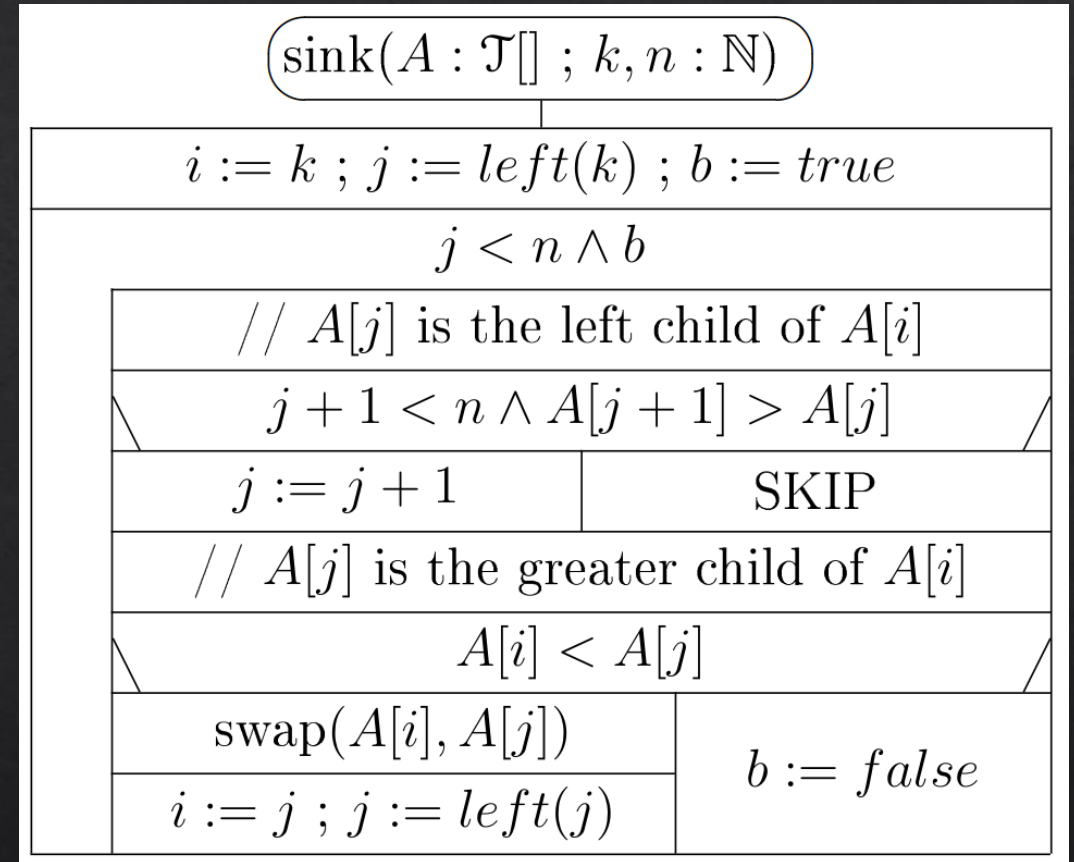
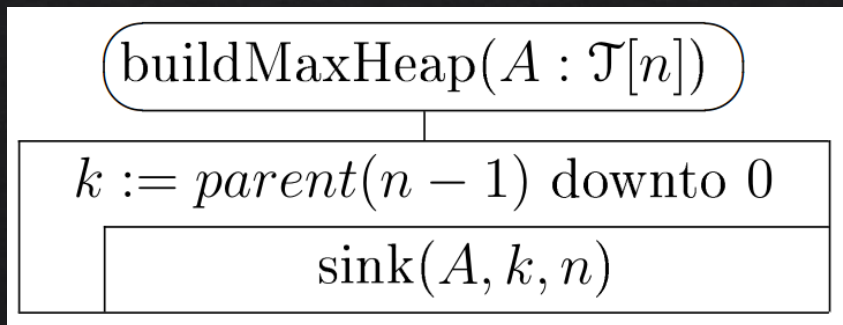
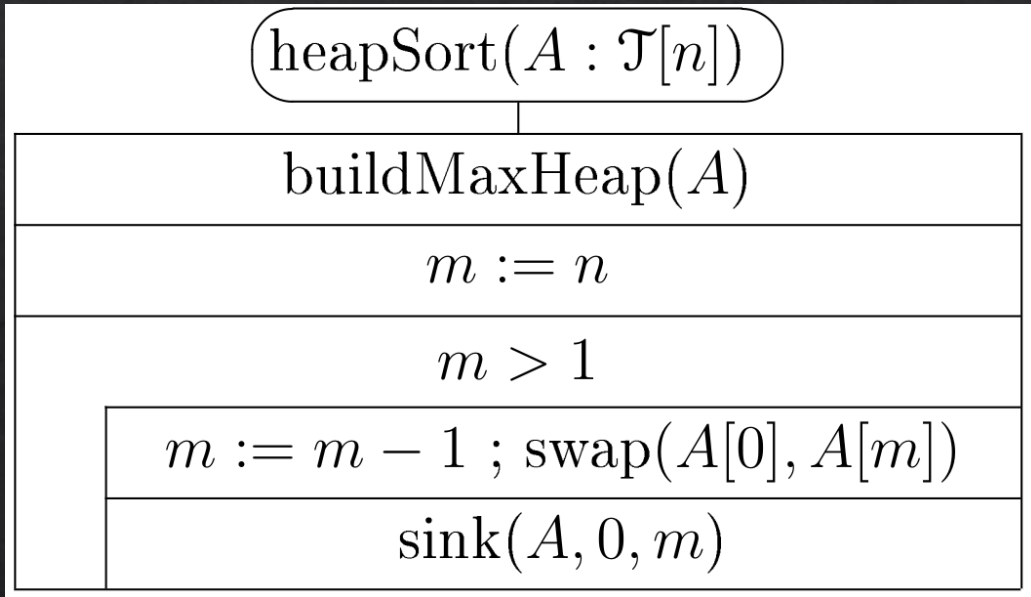
- Build a heap
 - with $\text{buildMaxHeap} \rightarrow O(n)$ time
 - we could do it with n add procedures, but that would result in $\rightarrow O(n \log n)$ time

- Next we take the elements and place them in their desired place = n removals with sink operation $\rightarrow O(n \log n)$ time complexity

Time Complexity: $O(n \log n)$

In place sorting (no extra space needed).

Heap sort



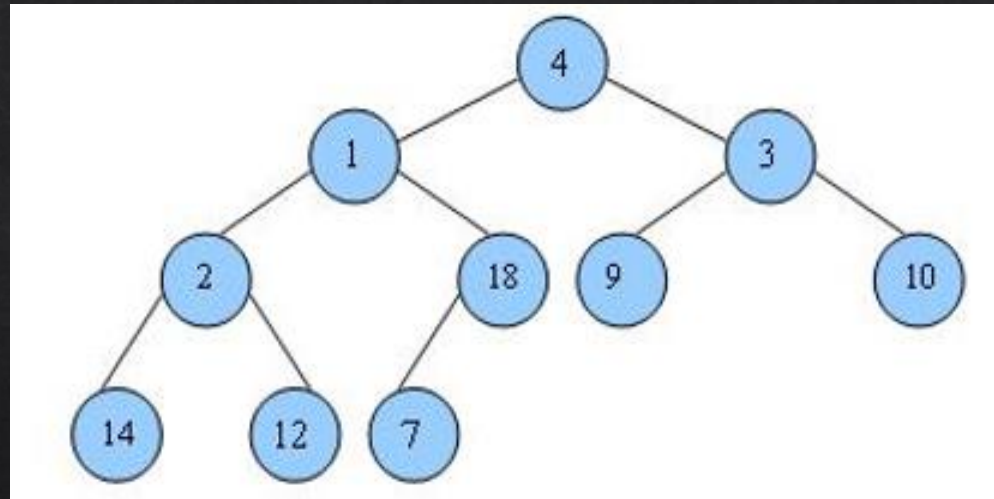
Time Complexity: $\Theta(n \log n)$

Exercise: demonstrate heap sort

◇ Input:

[4, 1, 3, 2, 18, 9, 10, 14, 12, 7]

◇ Presented as a complete binary tree:

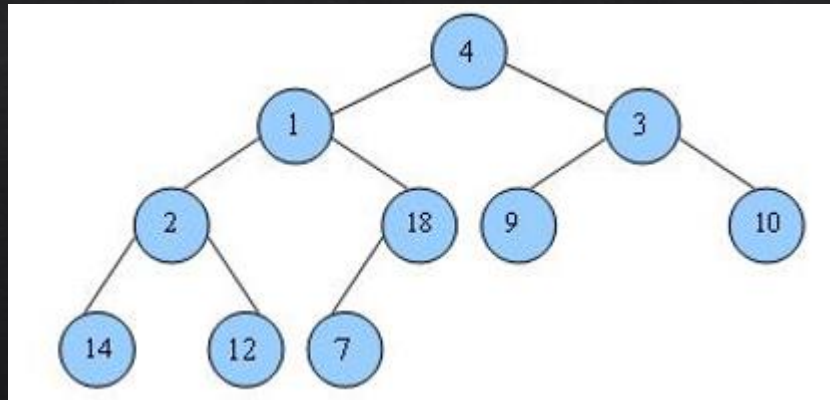


buildMaxHeap

$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n - 1) \text{ downto } 0$

$\text{sink}(A, k, n)$



➤ $n/2$ sink operations

➤ from $k := \left\lfloor \frac{n-1}{2} \right\rfloor$ to 0

Step 1.: Building max heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7
k					$n-1$				

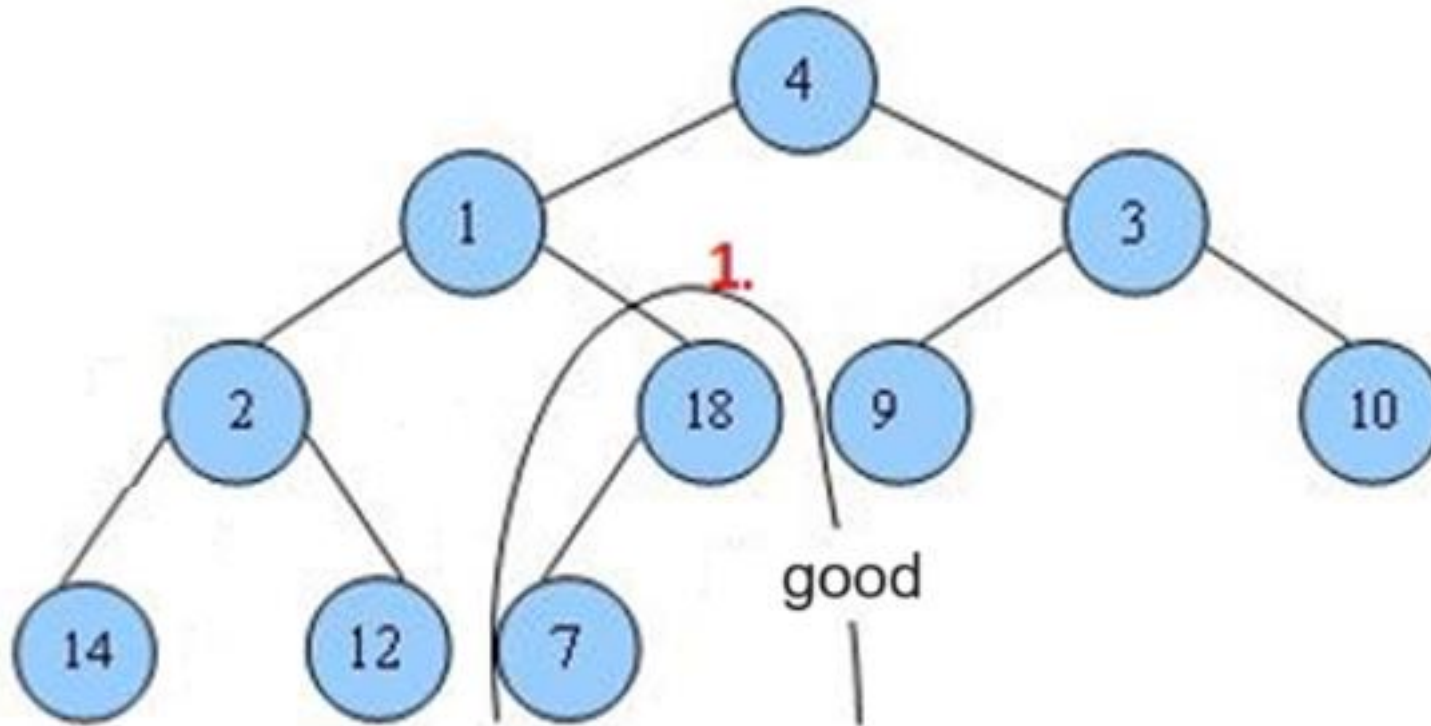
$$k := \left\lfloor \frac{n-1}{2} \right\rfloor = 4$$

1.
 $k=4$
 $\text{Sink}(A, 4, 10)$

$\text{buildMaxHeap}(A : \mathcal{T}[n])$

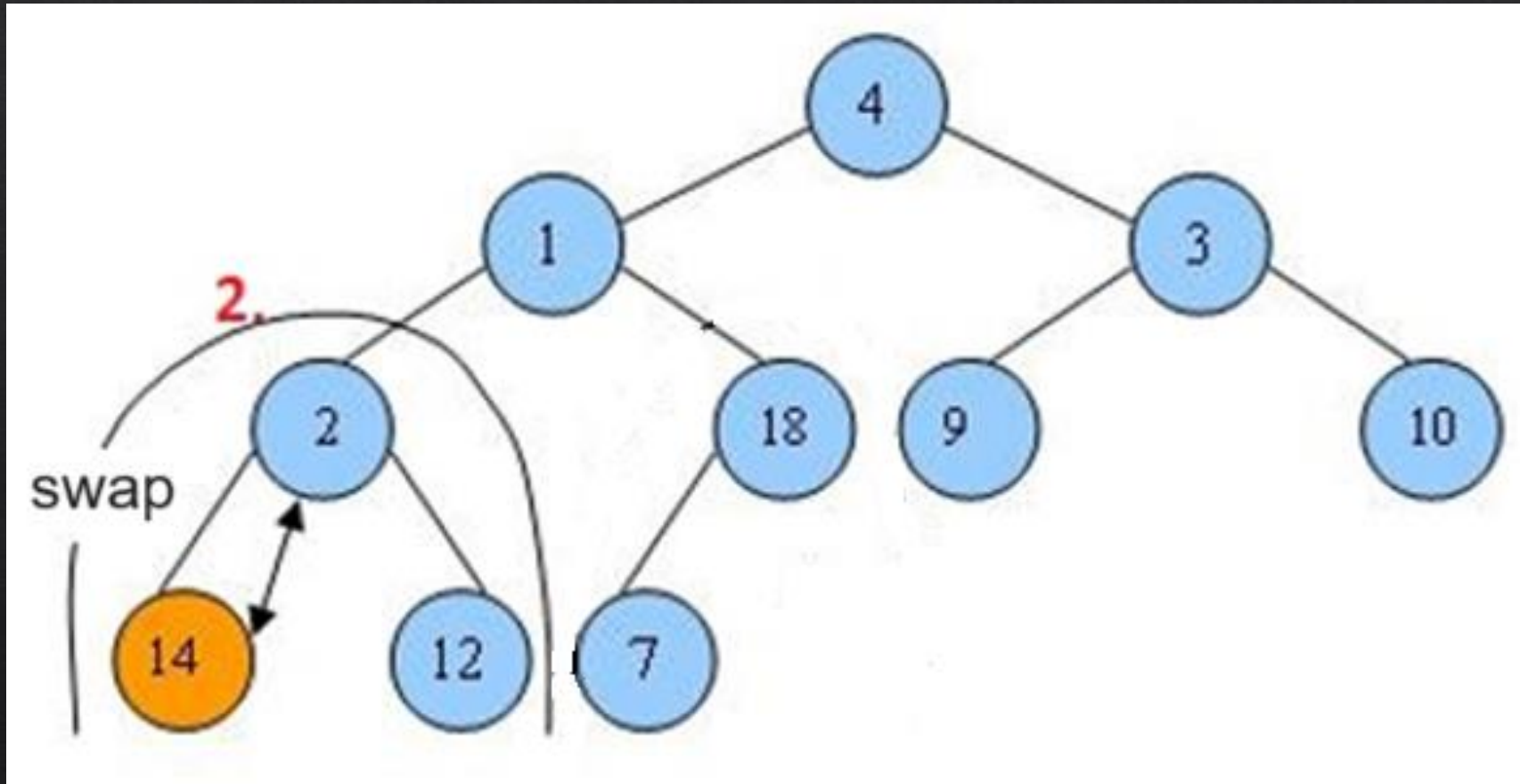
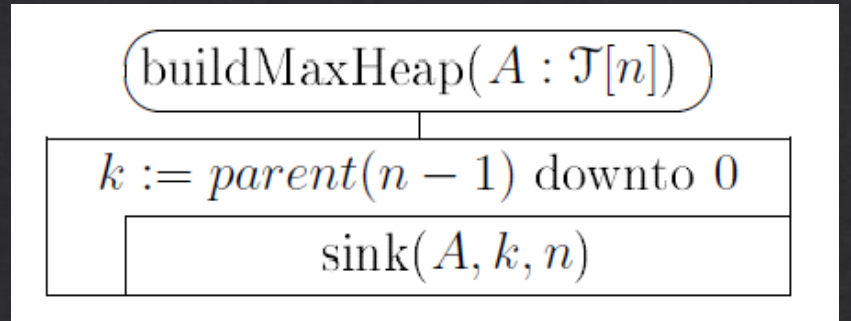
$k := \text{parent}(n-1)$ downto 0

$\text{sink}(A, k, n)$



Building max heap

2.
k=3
Sink(A,3,10)



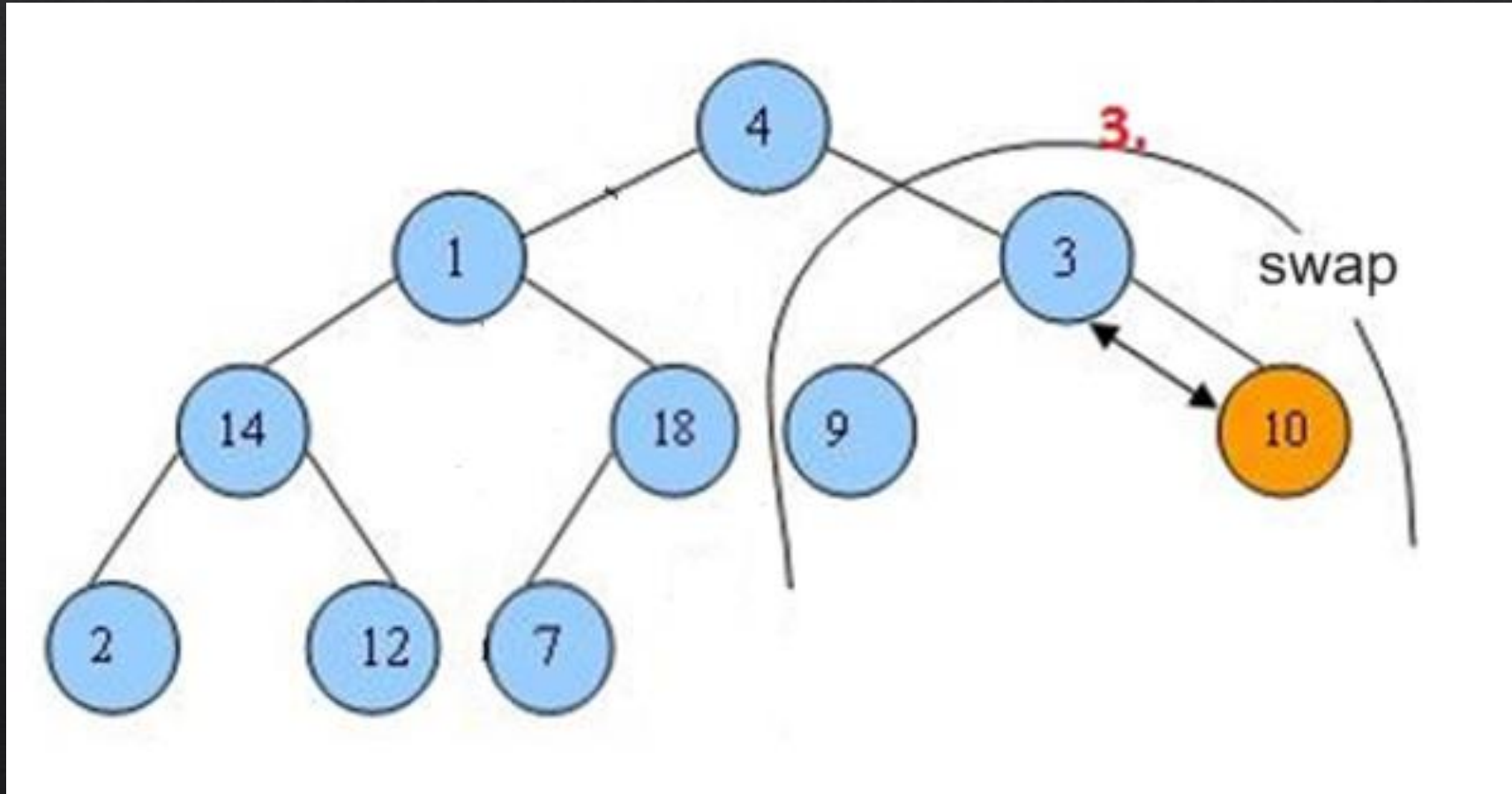
Building max heap

3.
k=2
Sink(A,2,10)

buildMaxHeap($A : \mathcal{T}[n]$)

$k := \text{parent}(n - 1)$ downto 0

 sink(A, k, n)



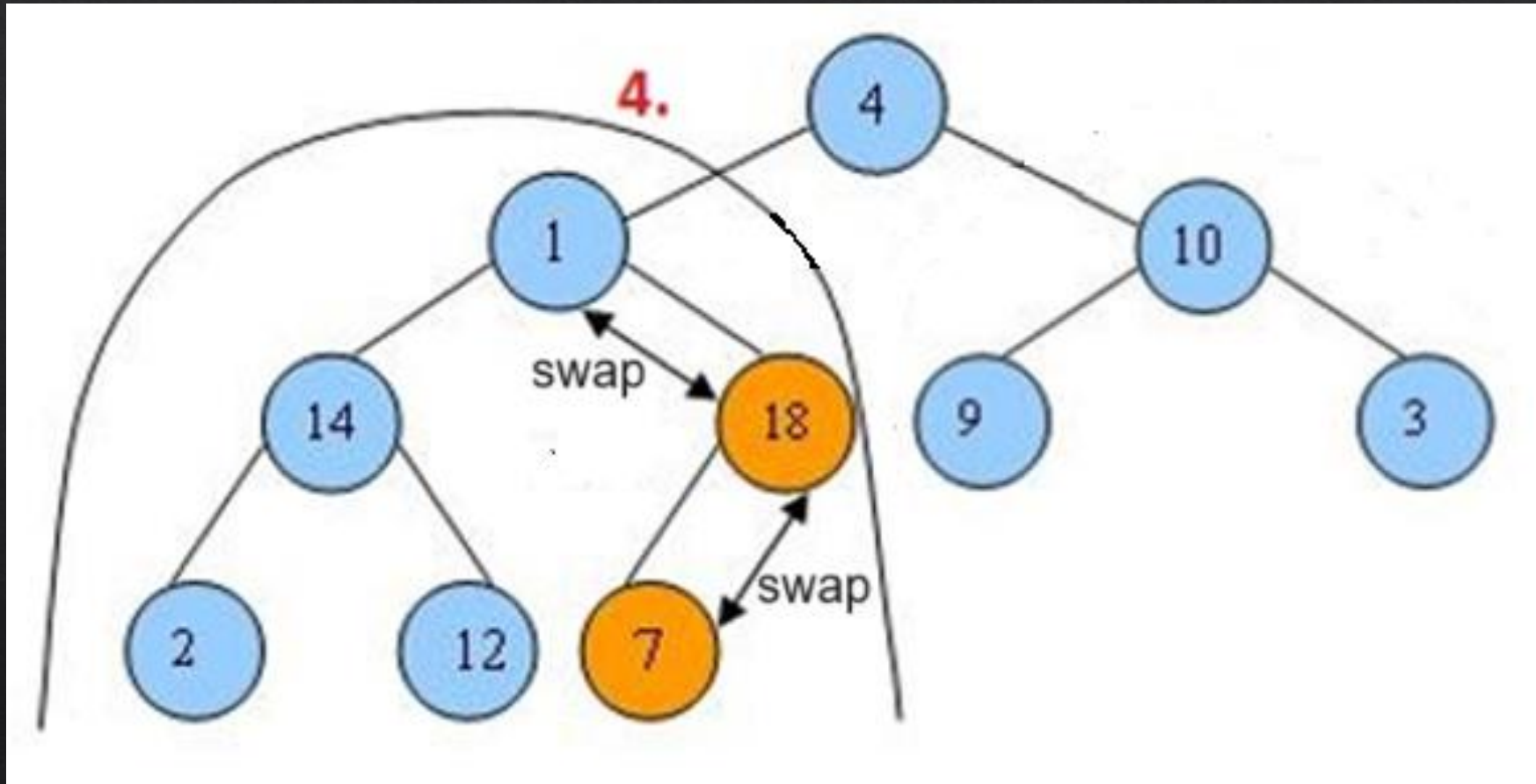
Building max heap

4.
k=1
Sink(A,1,10)

$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n - 1) \text{ downto } 0$

$\text{sink}(A, k, n)$



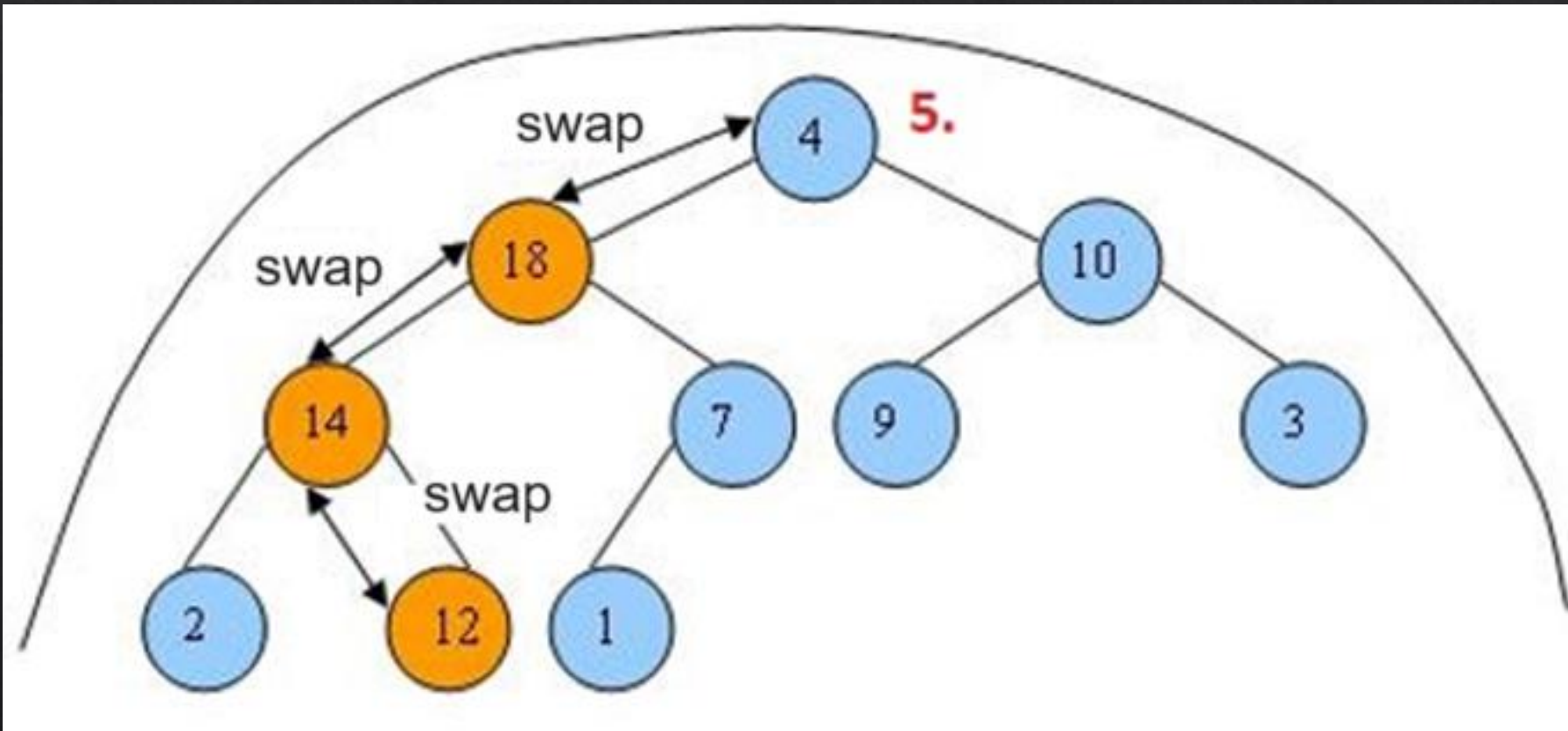
Building max heap

5.
k=0
Sink(A,0,10)

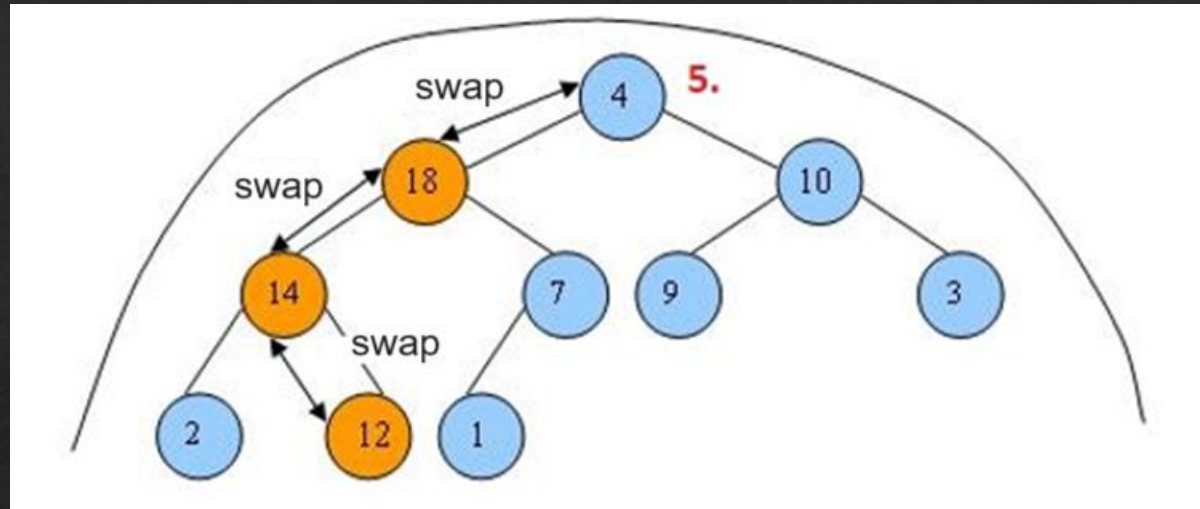
$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n - 1) \text{ downto } 0$

$\text{sink}(A, k, n)$

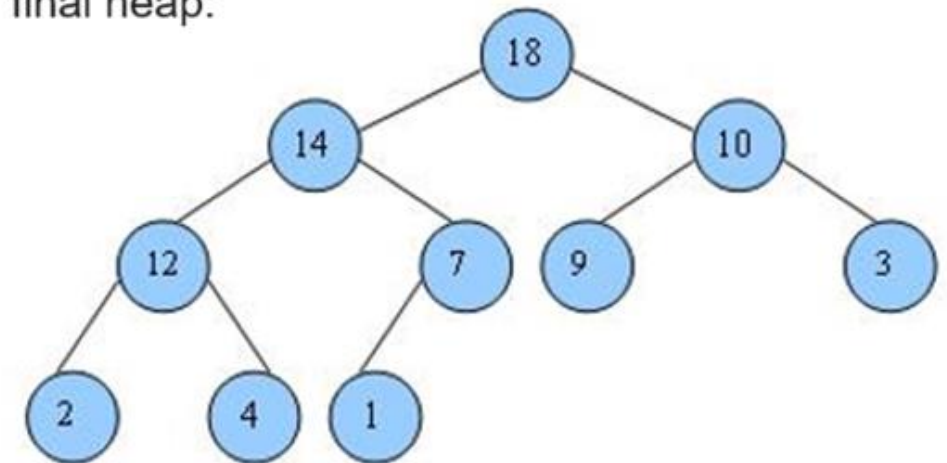


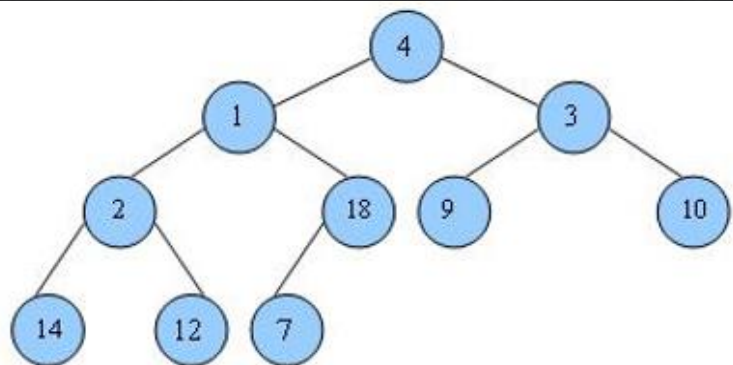
◇ Last sink in detail:



0	1	2	3	4	5	6	7	8	9
4	18	10	14	7	9	3	2	12	1
18	4	10	14	7	9	3	2	12	1
18	14	10	4	7	9	3	2	12	1
18	14	10	12	7	9	3	2	4	1

final heap:



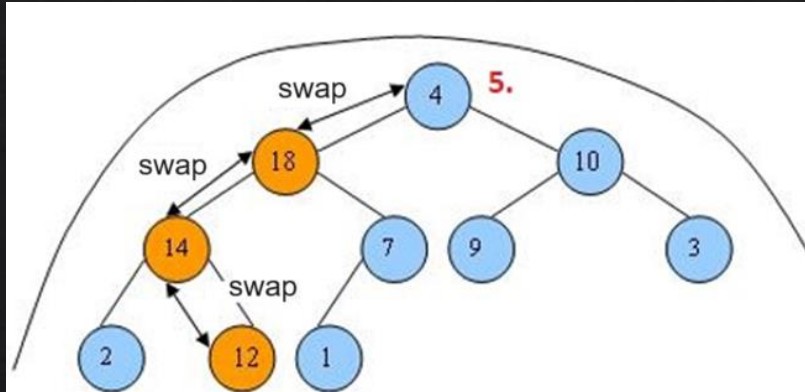
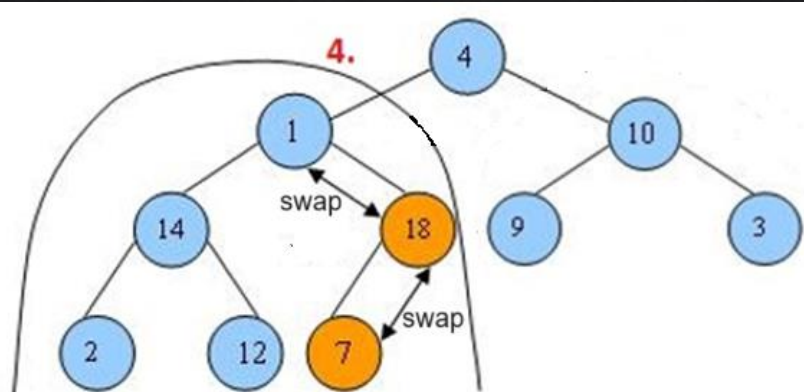
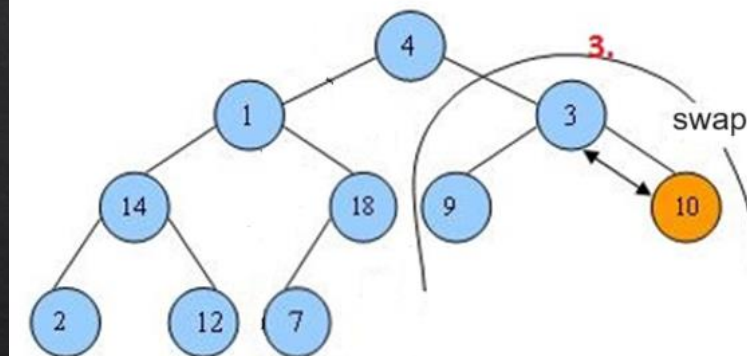
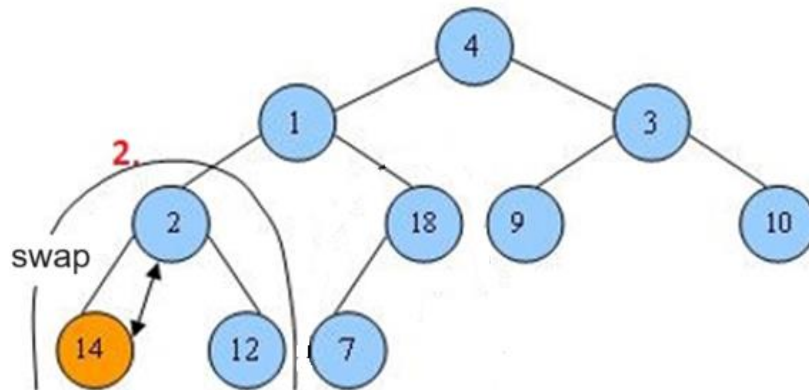
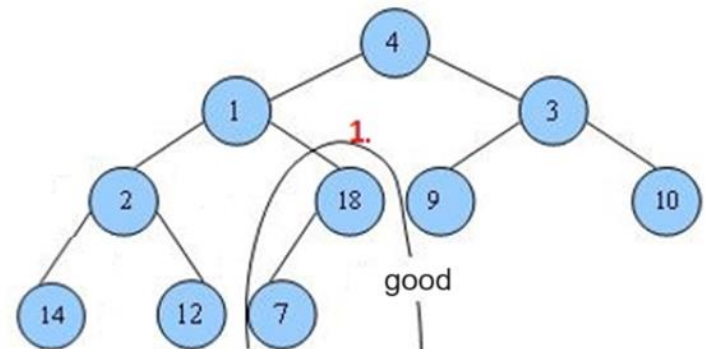


Building max heap

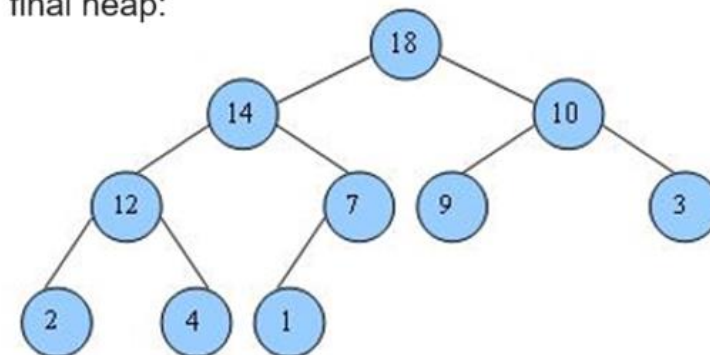
buildMaxHeap($A : \mathcal{T}[n]$)

$k := \text{parent}(n - 1)$ downto 0

sink(A, k, n)



final heap:



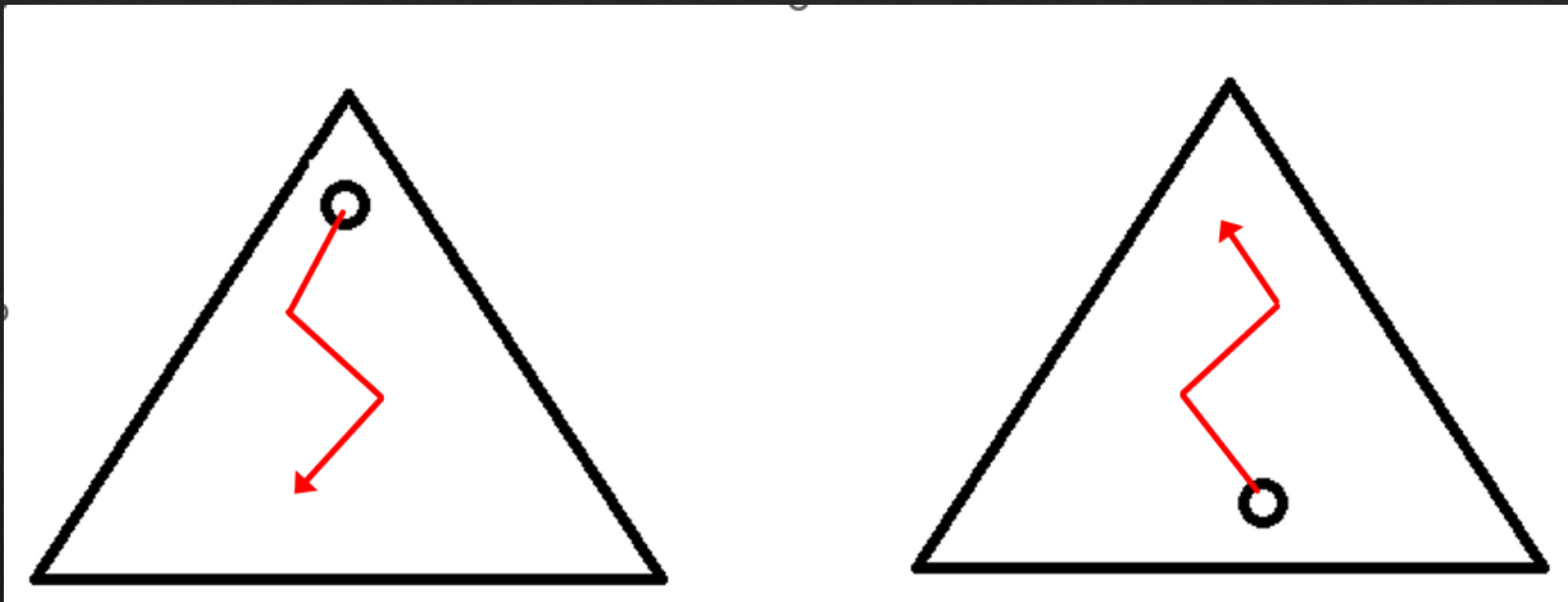
buildMaxHeap time complexity

Time complexity of buildMaxHeap:

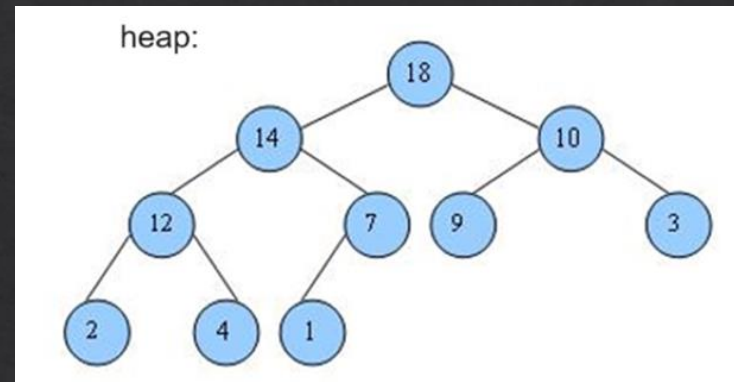
- ◇ Easy to show: $O(n \log n)$
- ◇ **Harder to show, but true: $\Theta(n)$**
- ◇ **Important:**
 - ◇ Instead of building the heap from bottom up with sinks we could use PrQueue insertions:
for $i = 1..n$ perform $\text{add}(A[i])$
 - ◇ in this case every element is added at the end, and then heaps-up (rising procedure)
 - ◇ But:

We use buildMaxHeap, because it gives better time complexity!

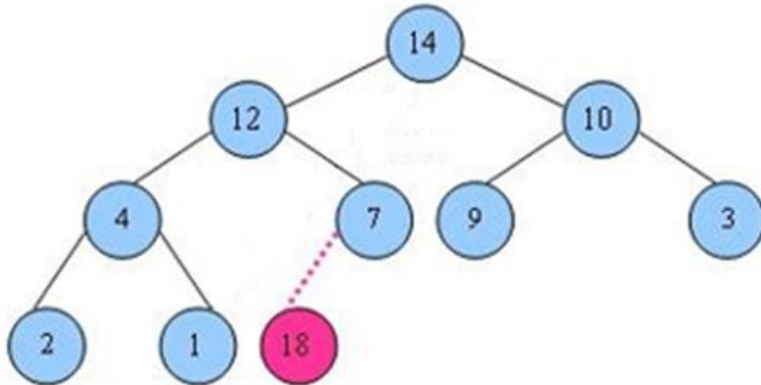
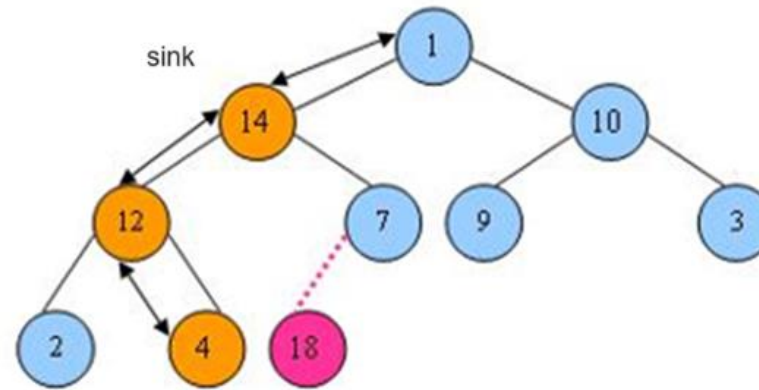
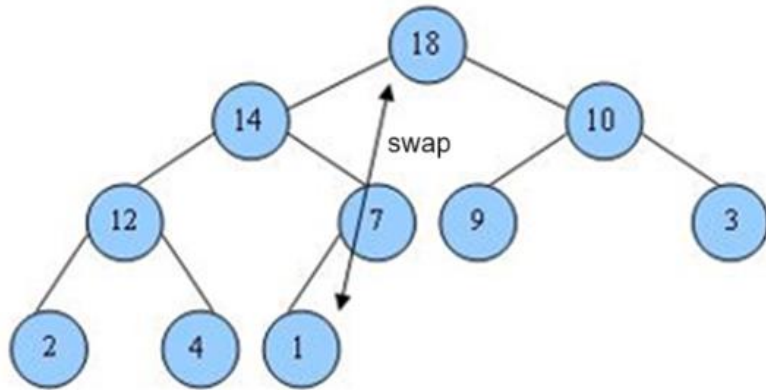
- ◇ With buildMaxHeap you go from the bottom level up to the top, and you might be making longer and longer sink procedures, but for less and less elements (half as much as on the previous level). This leads to $\Theta(n)$ time complexity.
- ◇ Adding the elements one by one with add procedures is like building the heap from top level to bottom: in this case the number of elements to be risen is more and more (double), and the length of the rising procedure can also be more and more (if larger elements added), so this can lead to $O(n \log n)$ time complexity in worst case.



Step 2. Sorting

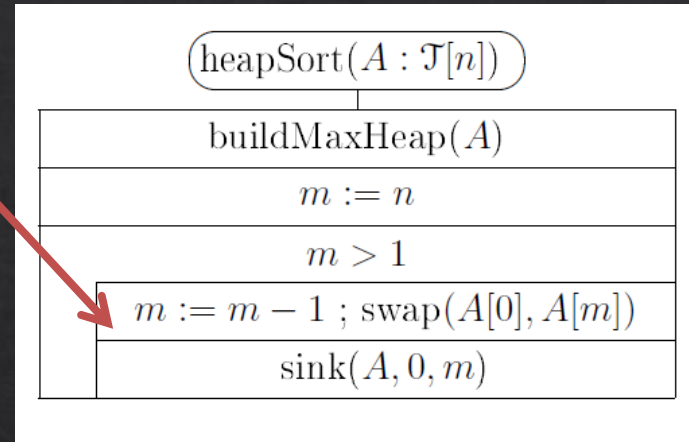


◇ We will only present two steps of this sorting part.



By the end of the first iteration, the heap structure is good, and the second largest element (14) is at the top. 18 is still in the array, but logically not part of the heap.

0	1	2	3	4	5	6	7	8	9
14	12	10	4	7	9	3	2	1	18



$m=10 \rightarrow m=9$

$\text{swap}(A[0], A[9])$

$\text{sink}(A, 0, 9)$

heapSort($A : \mathcal{T}[n]$)

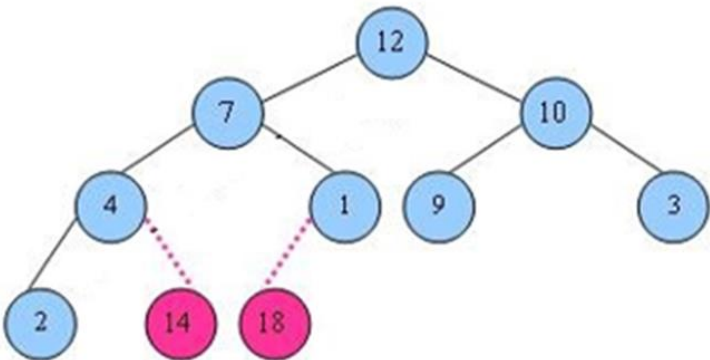
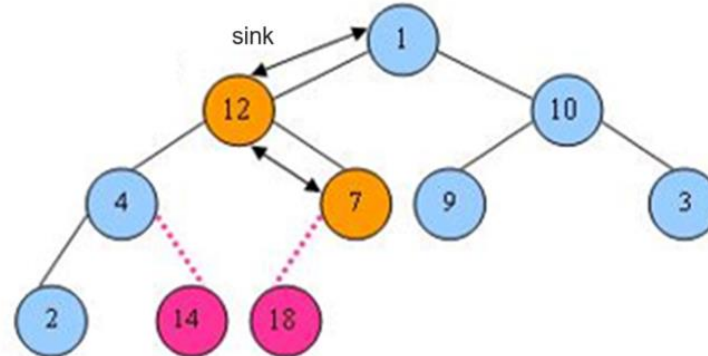
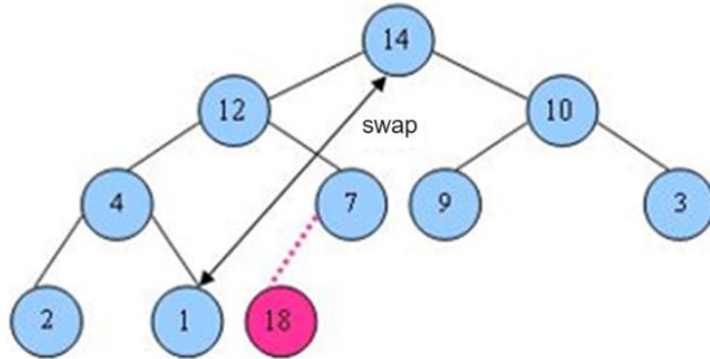
buildMaxHeap(A)

$m := n$

$m > 1$

$m := m - 1$; swap($A[0], A[m]$)

sink($A, 0, m$)



0	1	2	3	4	5	6	7	8	9
12	7	10	4	1	9	3	2	14	18

$m=9 \rightarrow m=8$

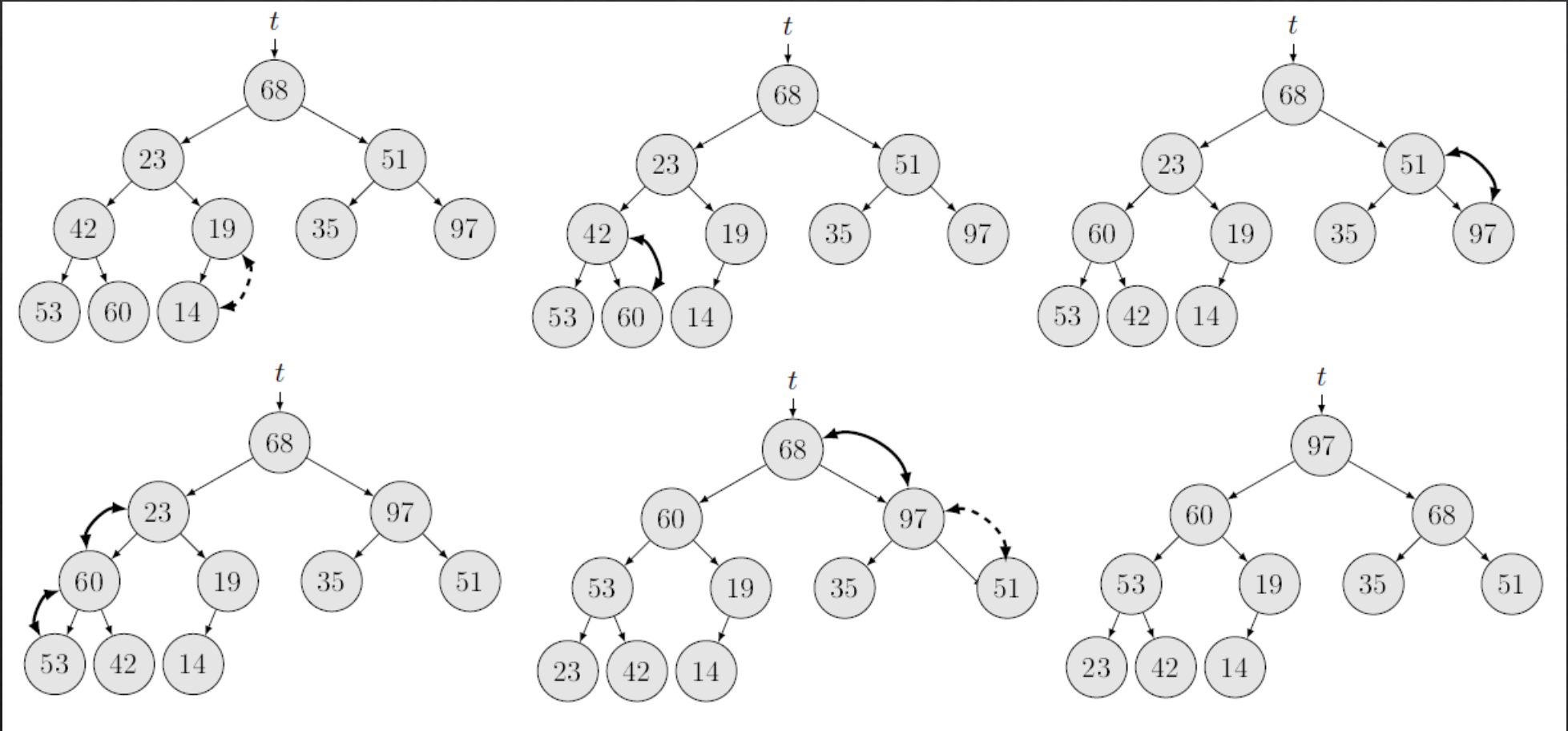
swap($A[0], A[8]$)

sink($A, 0, 8$)

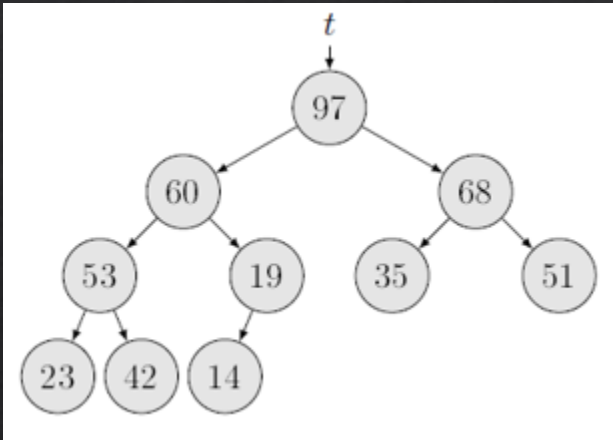
We will stop here, but as an exercise you should perform 2 more (or all the remaining) iterations.

Demonstration of heapsort in lecture notes on array [68, 23, 51, 42, 19, 35, 97, 53, 60, 14]

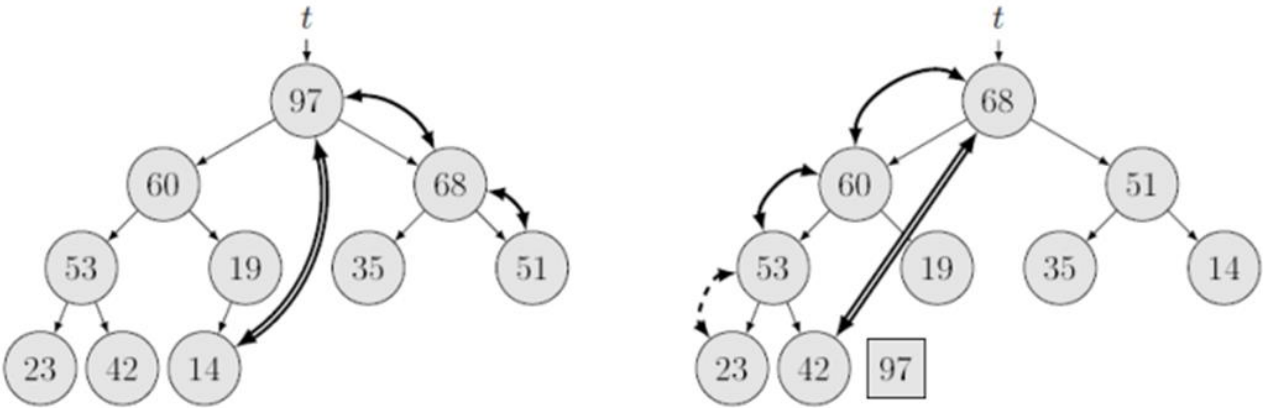
Step 1.: buildMaxHeap:



Step 2. the sorting:

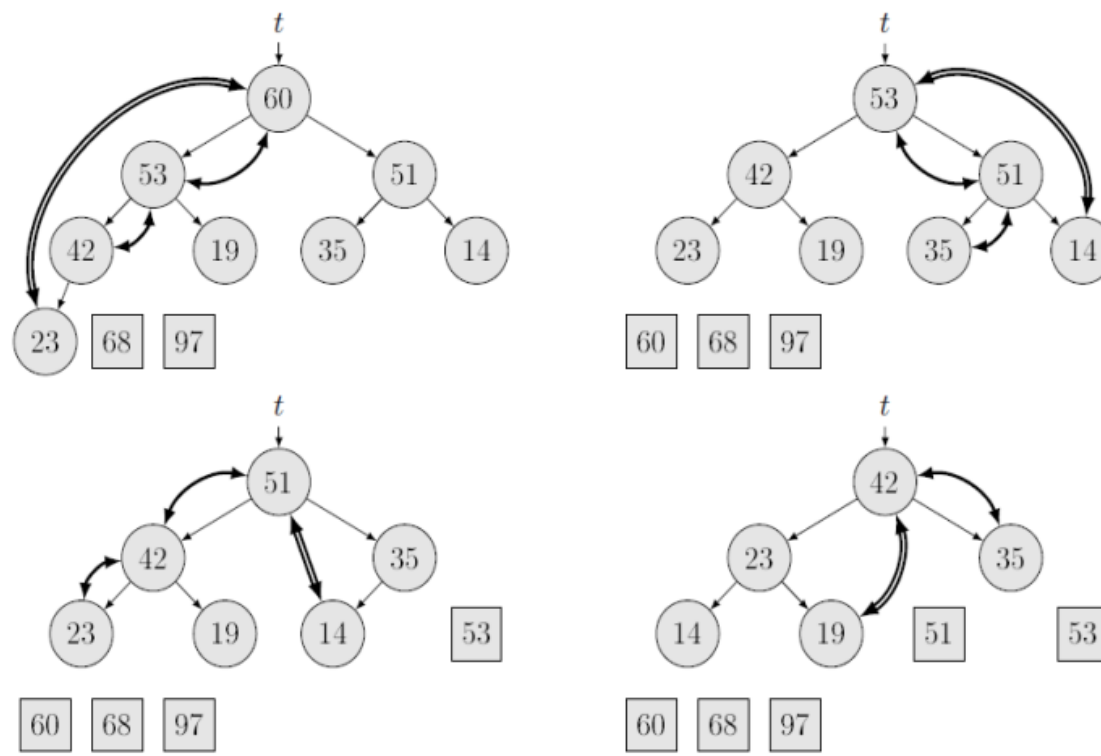


Heap sort visualization based on Figure 24. Here, we start with the result of buildMaxHeap(). Double arrows represent the swap of the largest element with the last element of the heap, fitting into its desired place. The subsequent arrows demonstrate the sinking process after the initial swap. Dashed arrows compare where the sinking ends because no swap is needed.



swap	~97	60	68	53	19	35	51	23	42	~14
sink	*14	60	#68	53	19	35	51	23	42	+97
...	68	60	*14	53	19	35	#51	23	42	+97
swap	~68	60	51	53	19	35	14	23	~42	+97
sink	*42	#60	51	53	19	35	14	23	+68	+97
...	60	*42	51	#53	19	35	14	23	+68	+97
.	60	53	51	*42	19	35	14	#23	+68	+97

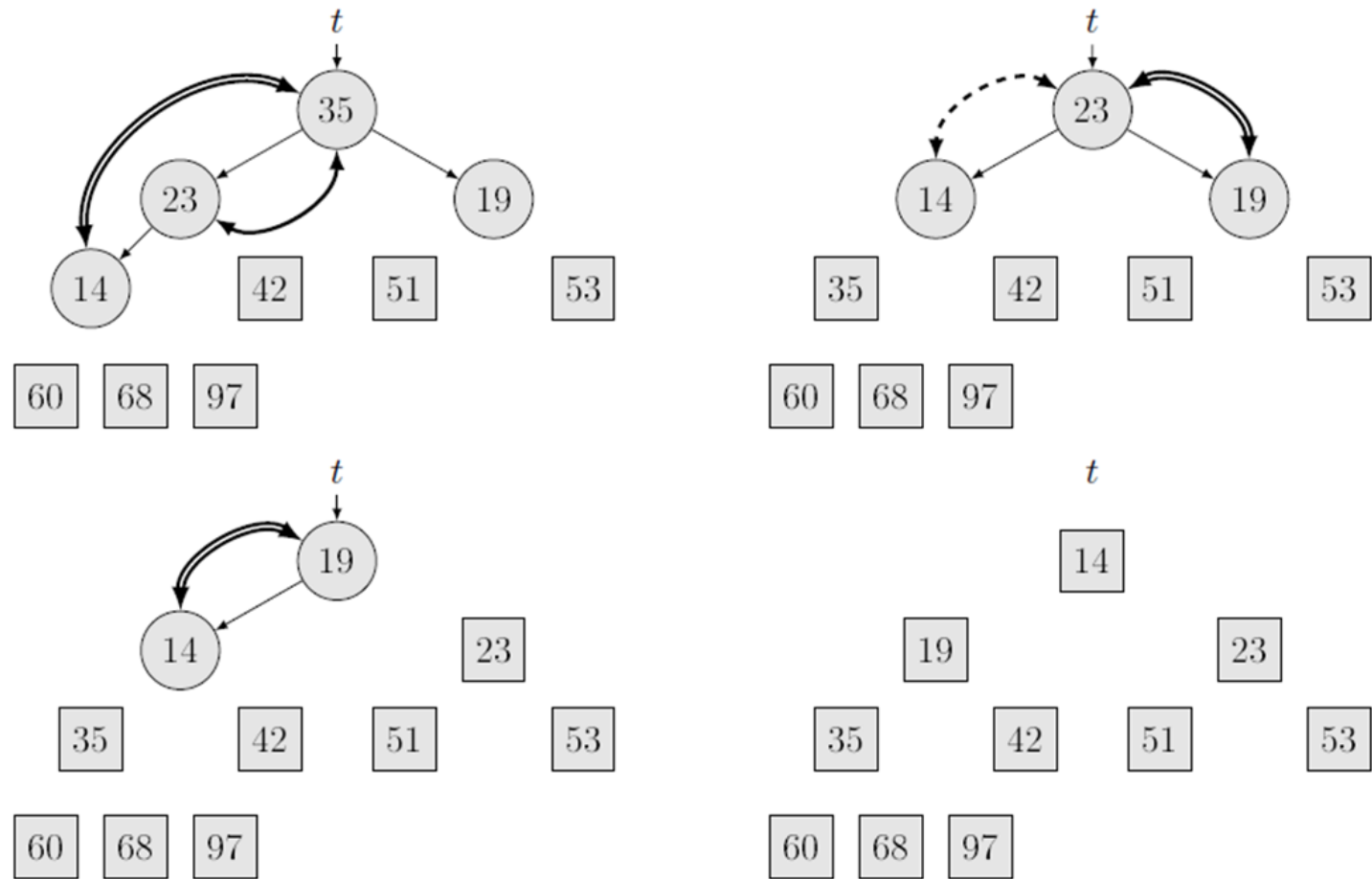
- * parent
- # greater child
- ~ element to be swapped



* parent
 # greater child
 ~ element to be swapped

swap	~60	53	51	42	19	35	14	~23	+68	+97
sink	*23	#53	51	42	19	35	14	+60	+68	+97
...	53	*23	51	#42	19	35	14	+60	+68	+97
swap	~53	42	51	23	19	35	~14	+60	+68	+97
sink	*14	42	#51	23	19	35	+53	+60	+68	+97
...	51	42	*14	23	19	#35	+53	+60	+68	+97
swap	~51	42	35	23	19	~14	+53	+60	+68	+97
sink	*14	#42	35	23	19	+51	+53	+60	+68	+97
...	42	*14	35	#23	19	+51	+53	+60	+68	+97
swap	~42	23	35	14	~19	+51	+53	+60	+68	+97
sink	*19	23	#35	14	+42	+51	+53	+60	+68	+97
swap	~35	23	19	~14	+42	+51	+53	+60	+68	+97
sink	*14	#23	19	+35	+42	+51	+53	+60	+68	+97

* parent
 # greater child
 ~ element to be swapped



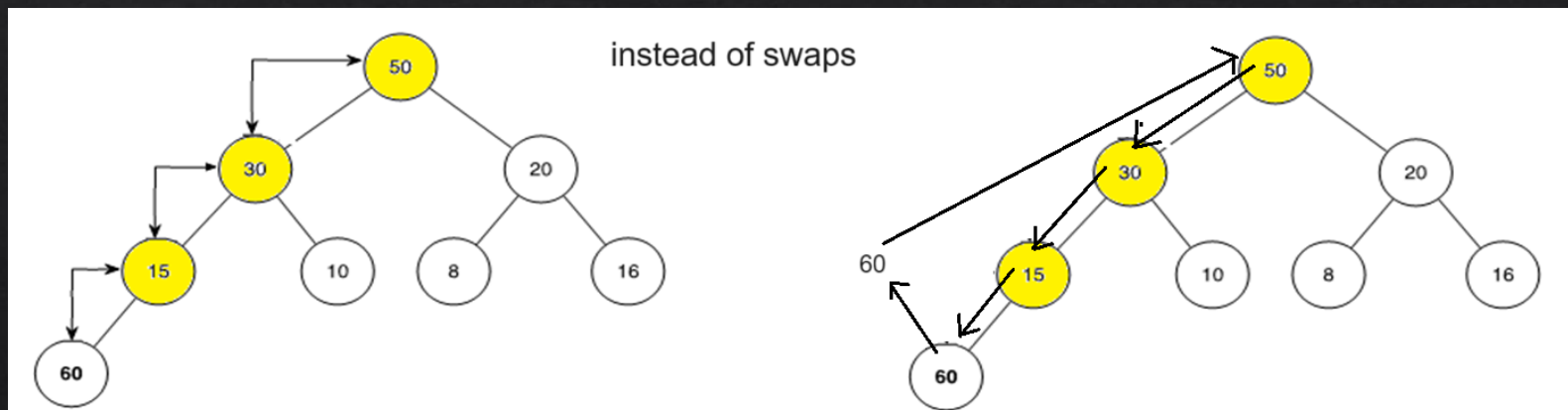
swap	~35	23	19	~14	+42	+51	+53	+60	+68	+97
sink	*14	#23	19	+35	+42	+51	+53	+60	+68	+97
swap	~23	14	~19	+35	+42	+51	+53	+60	+68	+97
sink	*19	#14	+23	+35	+42	+51	+53	+60	+68	+97
swap	~19	~14	+23	+35	+42	+51	+53	+60	+68	+97
sink	*14	+19	+23	+35	+42	+51	+53	+60	+68	+97
.	+14	+19	+23	+35	+42	+51	+53	+60	+68	+97

Optional assignment

Using swaps in the sink (heap-down) and rise (heap-up) procedures is not too efficient. You are making too many overwriting in the array, the element to be moved may be copied $\log n$ times. Improve the efficiency of these methods:

1. Implement the sink($A:T[];k,n:N$) procedure without the use of direct swaps. Instead copy the element to be sinked into a helper variable, then „sink” the „hole” which has been created, by copying the other elements only.
2. Implement the rising of an element in a similar way.

Rising:



rise or up-heap operation

rise ($A: T[n], j:N$)

$i := \text{parent}(j)$	
$j > 0 \wedge A[i] < x$	
	$\text{swap}(A[i], A[j])$
	$j := i ; \quad i := \text{parent}(i)$

OHW

1. Given $A[1..n]$ array, ascendingly ordered. Write a recursive algorithm building an 'optimal' binary tree. Let's allow parent pointers (Node3*). Let's return the root.
2. Implement the Sink($A:T[];k,n:N$) without direct swaps: Use a single helper variable in which we store the element to be sinked, and each time just copy one of the elements instead of a full swap.